

PRALLO – Théo Minger

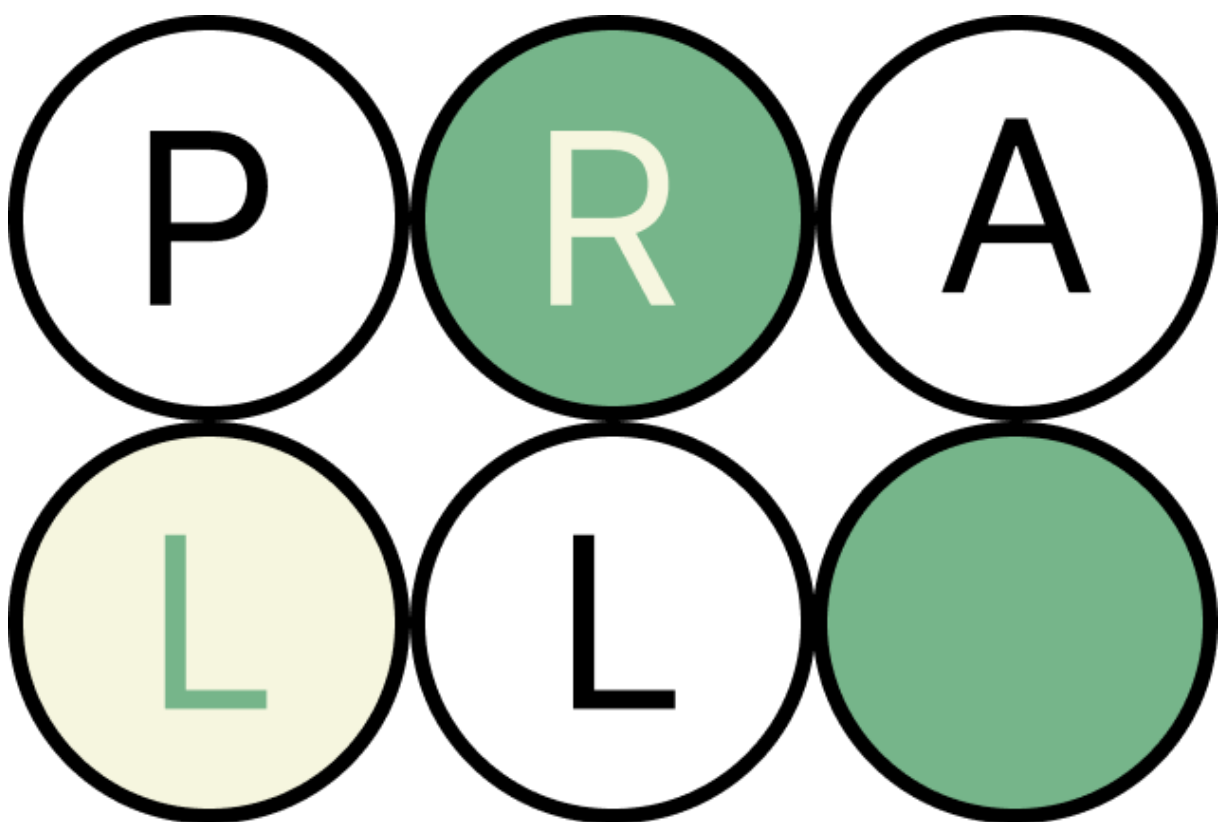


Table des matières

1	Analyse préliminaire	3
1.1	Introduction	3
1.1.1	Résumé	3
1.1.2	Prérequis	4
1.2	Objectifs.....	4
1.2.1	Objectifs de formation.....	4
1.2.2	Objectifs du produit.....	4
1.3	Gestion de projet	5
1.3.1	Outils	5
1.3.2	Pratiques	5
1.3.3	Project-Handbook.....	6
1.3.4	Personnes impliquées et rôles.....	6
1.4	Planification initiale	6
2	Analyse / Conception.....	8
2.1	Lexique	8
2.2	Contexte produit	8
2.3	Contextes techniques	9
2.3.1	Opérationnel	9
2.3.2	Staging	9
2.3.3	Développement.....	10
2.3.4	Technologies utiles :	10
2.4	L'architecture et les services en schéma	11
2.4.1	Schéma de l'environnement de développement.....	11
2.4.2	Schéma de l'environnement de staging	12
2.4.3	Digramme d'état	13
2.4.4	Diagramme de flux concernant l'authentification	15
2.5	Analyse fonctionnelle.....	16
2.5.1	Réalisation de maquettes	16
2.5.2	Discussion avec le Client/Product Owner	16
2.5.3	User story	17
2.6	Stratégie de test.....	18
2.7	Risques techniques	19
2.8	Planification	19
3	Réalisation.....	20
3.1	Déroulement.....	20
3.1.1	Prompt Bullet	20
3.1.2	MCD et MLD.....	21

3.1.3	Feature/Bugfix	24
3.1.4	Tests.....	25
3.1.5	Mise en Staging	26
3.2	Sprints	27
3.2.1	Fin du premier Sprint, 26.05.2026	27
3.2.2	Fin du deuxième sprint, 28.05.2026.....	28
3.3	Description des tests effectués	29
3.4	Bilan.....	32
3.4.1	Erreurs restantes	32
3.4.2	Sprints	33
3.4.3	Dette technique.....	33
3.5	Recours à l'intelligence artificielle	34
3.6	Liste des documents fournis	36
4	Conclusions	36
4.1	Objectifs.....	36
4.2	Difficultés particulières	36
4.3	Suites possibles pour le projet.....	36
4.4	Avis personnel	37
5	Annexes.....	37
5.1	Sources – Bibliographie.....	37
5.2	Figures.....	38
5.3	Journal de travail	39
5.4	Manuel d'utilisation	42
5.5	Maquettes.....	45
5.6	Prompt-bullet	56

1 Analyse préliminaire

1.1 Introduction

1.1.1 Résumé

Chaque année, lors de leur 4ème année, les élèves en informatique de l'ETML se voient attribuer un projet « P_APPRO ». En effet, avant de commencer le TPI, ceux-ci doivent se réhabituer aux pratiques utiliser sur place. Pour se faire, chaque étudiant reçoit un projet à réaliser et un professeur référant. Mais, comment cela se passe-t-il ? En effet, l'attribution d'un projet et d'un professeur par élèves est parfois chaotique. Cela dépend de beaucoup de chose : des envies de l'élèves, des facultés enseignées par les professeurs, du nombre d'élèves par professeur et j'en passe ...

C'est ici que **Prallo** entre en jeu ! **Prallo** est une application web qui permet de rendre ce moment chaotique un peu plus simple.

1.1.2 Prérequis

Voici une liste des cours réalisés pour permettre la bonne réalisation du projet :

• C294 – Réaliser le front-end d'une application Web interactive
• C295 – Réaliser le back-end pour des applications
• P_WEB – projet Web
• P_BULLE – Snake
• I162 – Analyser et modéliser des données
• I164 – Créer des bases de données
• I165 – Utiliser des bases de données NoSQL
• C106 – Interroger, traiter et assurer la maintenance des bases de données
• P_DB – Projet base de données

1.2 Objectifs

1.2.1 Objectifs de formation

L'intention de ce projet est le suivant : Réaliser un UX/UI user friendly pour une application web complexe qui pourrait être utilisée par n'importe quelle école/entreprise, pour ensuite la pousser en production. Ce que ce projet m'apporte :

- **Gestion de projet** : je suis capable de gérer mon propre projet, en utilisant des méthodes agiles. J'utilise des user stories pour clarifier les objectifs. Je manipule Github pour avoir un suivi du code et un journal de travail pour suivre mes démarches
- **Design UI/UX** : je me renseigne sur des principes d'UI et d'UX pour réaliser une interface user friendly. Je réalise par conséquent des maquettes haute-fidélité et des user story complètes
- **Architecture logicielle** : je choisis mon environnement et technologie en fonction des besoins.
- **Développement/Staging** : je mets en place un environnement de développement sain et un environnement de staging. Je me renseigne donc sur les méthodes à ma disposition pour mettre en staging l'application web et les mets en pratique.

1.2.2 Objectifs du produit.

Les objectifs de produit sont :

- **Authentification** : Prallo contient 4 rôles : la Gouvernance ou doyen, professeur, étudiant et invité. Pour se faire un moyen d'authentification nous permet de savoir qui a quel rôle.
- **Annoncer les compétences** : les professeurs sont capables d'enregistrer leurs technologies favorites.
- **Nombre de projets** : les Doyens assignent un nombre max de projet par professeurs
- **Liste de projets** : Les professeurs peuvent créer des projets. Ceux si sont visibles par les étudiants dans une liste quand l'état de l'application le permet.
- **Voter pour des projets** : les élèves votent pour des projets qui leurs donnent envie quand l'état de l'application le permet
- **Gérer la répartition** : un algorithme de tri fais la répartition des projets à l'aide d'un indice de satisfaction.
- **Gérer l'état du système** : les doyens gèrent l'état du système. Il en a 5 : Préparation, Repérage, Vote, Répartition et Publication

1.3 Gestion de projet

1.3.1 Outils

Pour la gestion de ce projet, les outils sont les suivants :

- **Journal de travail** : la réalisation d'un journal de travail sous format Excel permet de suivre mes démarches
- **Git Project** : permet de créer des User stories exhaustives dans un Kanban et aide pour le suivi du développement.
- **GitHub** : Permet de contrôler les versions avec des release

1.3.2 Pratiques

Le projet est conduit selon les **méthodologies agiles**, privilégiant l'adaptation continue et la communication transparente. Dans ce cadre, des échanges réguliers avec le client sont essentiels pour absorber les nombreuses itérations possibles.

Comme mentionné précédemment, l'utilisation de **GitHub Projects** permet de :

- **Implémenter un tableau Kanban** : pour une visualisation en temps réel de l'avancement des tâches et une gestion des priorités.
- **Établir une "Definition of Done" (DoD)** : pour garantir que chaque fonctionnalité répond à des standards de qualité stricts (tests, revues de code) avant d'être considérée comme terminée.
- **Gérer le cycle de vie des User Stories** : chaque fonctionnalité est rédigée sous forme de User Story afin de se concentrer sur la compréhension entre le client et moi.

Ces **User Stories** servent de base de négociation et de compréhension commune. Elles font l'objet d'une **revue systématique avec le chef de projet et le client** avant leur intégration au backlog de développement. Ce processus de validation itératif permet de s'assurer que les spécifications définies correspondent parfaitement aux attentes métier avant de lancer les développements techniques.

1.3.3 Project-Handbook

Durant ce projet, le respect des règles disponibles dans le repository GitHub "[Project Handbook](#)" **font foi**. Voici quelques règles notables :

L'analyse fonctionnelle est réalisée au travers d'une liste de User Stories (US), comportant :

- *Un titre*
- *Une description ("En tant que ... je veux ... pour ...")*
- *Un ou plusieurs éléments graphiques (maquette, schéma, diagramme,...)*
- *Un nombre significatif de tests d'acceptance*

Ou encore, concernant GitHub :

- *Créez un repo privé dans votre compte et invitez-moi en lecture seule.*
- *Suivez la convention de nommage des commits [conventional commits](#).*
- *Appliquez le modèle [gitflow](#) pour la gestion des branches, avec les branches main, develop et les branches de features.*
- *Lors des sprint reviews, les fonctionnalités sont évaluées dans leur branche de feature. Si la story est validée, la branche de feature est fermée.*
- *Tous les fichiers utiles au travail et autres que du code (documentation technique interne ou externe, rapports, exemples, snippets, etc...) se trouvent dans l'arborescence sous doc, dont la structure est livrée à votre bon sens.*
- *Bien que cela soit une pratique controversée, je demande à ce que les fichiers concernant la documentation (notamment le rapport et le journal de travail) soient également intégrés dans le repository, dans le dossier doc.*

1.3.4 Personnes impliquées et rôles

- **Chef de projet, product owner et client** : Xavier Carrel
- **Expert 1** : Nicolas Borböen
- **Expert 2** : Nemanja Pantic
- **Apprenti** : Théo Minger

1.4 Planification initiale

Ce projet est réalisé en deux temps : une première partie qui consiste en la rédaction du rapport et du projet et une deuxième consacrée à la présentation.

La première partie durera du 07.05.2026 jusqu'au lundi 01.06.2026.

Une seconde partie concerne une soutenance qui aura lieu entre **le mercredi 10.06.2026 et le jeudi 11.06.2026.**

L'horaire est le suivant :

Lundi	8h-11h25	12h20-15h
Mardi	8h-11h25	13h10-16h35
Mercredi	8h-11h25	12h20-15h
Jeudi	8h-12h25	13h10-16h35
Vendredi	8h-12h15	13h10-16h35

Semaine du lundi 04.05.2026 : 13 périodes

Semaine du lundi 11.05.2026: 22 périodes

Semaine du lundi 18.05.2026: 40 périodes

Semaine du lundi 25.05.2026: 33 périodes

Semaine du lundi 01.06.2026: 4 périodes

Total : 90h.

Dates importantes : Ascension 14 mai, Pont de l'Ascension 15 mai, Pentecôte 25 mai

La planification est la suivante :

#	Objectifs	Nombres d'heures	Revue
1	Introduction + introduction du Rapport	6	Vendredi 08.05
2	Réalisation des maquettes + User Story (Analyse)	25	Lundi 18.05
3	Ecriture de la partie 2. Analyse / Conception du Rapport	13	Mercredi 20.05
4	Réalisation du code + Partie 3. Réalisation du Rapport	28	Mercredi 27.05
5	Réalisation des tests E2E + Fonctionnels	5	Mercredi 27.05
6	Mise en Production	10	Vendredi 29.05
7	Conclusion	3	Lundi 01.06
	Total	90	

Dans le cahier des charges il est indiqué :

Analyse 25%, Implémentation 45%, Tests 10%, Documentation 10%. Néanmoins en vue de la nature de ce projet, la partie analyse et documentation sont censées prendre plus de temps, le planning en pourcentage est donc le suivant :

Analyse	Implémentation	Test	Documentation
30%	30%	10%	30%

2 Analyse / Conception

2.1 Lexique

- **Une compétence/technologie** : Il s'agit d'un savoir-faire technique, d'un outil ou d'un langage de programmation maîtrisé par un utilisateur ou requis pour la réalisation d'un projet au sein de l'application (par exemple : React, Gestion de projet, Python, Méthodes Agiles).
- **End to End (E2E)** : C'est un test qui simule le parcours complet d'un utilisateur réel. Au lieu de tester un petit morceau de code, on vérifie que l'ensemble du "voyage" (cliquer sur un bouton, se connecter, voir une page) fonctionne parfaitement du début à la fin.
- **Un état** : Dans PRALLO, il est question d'état d'application. Un état d'application représente la situation ou la configuration de l'application à un instant donné (par exemple : Préparation, Publication). Il détermine les actions possibles pour les utilisateurs et l'affichage des données.
- **MSAL** (Microsoft Authentication Library) : C'est la "boîte à outils" officielle de Microsoft pour permettre à vos utilisateurs de se connecter à votre application (via leur compte Microsoft) sans que vous ayez à gérer toute la partie complexe de la sécurité.
- **Un participant/invité** : Un participant ou invité est une personne qui s'est connecté à l'application PRALLO. Ce participant est ensuite enregistré dans la base de données. Le seul droit des participant est de visualiser les projets dans l'état « Repérage ». Une fois connecté, les doyens attribuent le rôle souhaité aux invités
- **PM2** : Gestionnaire de processus pour applications Node.js qui maintient vos services en arrière-plan. Il assure la haute disponibilité en redémarrant automatiquement votre application en cas de crash et facilite la gestion des logs.

2.2 Contexte produit

Chaque année, le mois de février marque une étape importante pour les étudiants de quatrième année de l'ETML en informatique. En effet, c'est dans ces dates que les étudiants reviennent d'une période de stage en entreprise. Afin de préparer au mieux leur Travail Personnel de Fin d'Études (TPI), la filière organise un projet d'approfondissement, nommé P_APPRO. Il s'agit d'un module qui prépare les élèves au TPI.

Cependant, la mise en place de ces projets individuels représente un défi logistique majeur pour la direction. La filière doit composer manuellement avec beaucoup de variables complexes : le nombre de projets maximales par enseignant, les compétences techniques spécifiques des professeurs, l'obligation d'attribuer certains projets prioritaires, et le respect des préférences exprimées par les élèves.

Harmoniser ces critères à la main s'avère extrêmement embêtant et il peut y avoir des erreurs de temps en temps.

C'est dans ce contexte qu'intervient l'application **Prallo**.

Prallo permet de mettre en relation les doyens, les professeurs et les élèves pour décider des projets mis à disposition, et des projets que les élèves vont réaliser.

2.3 Contextes techniques

2.3.1 Opérationnel

L'application Prallo est un site web classique accessible uniquement par internet. Elle ne s'installe pas sur les ordinateurs des utilisateurs. Elle est développée spécifiquement pour être utilisée sur un ordinateur de bureau (desktop) et n'est pas optimisée pour les téléphones ou les tablettes (pas de responsive). Tout le monde se connecte au même site via son navigateur, ce qui permet de centraliser toutes les données au même endroit.

2.3.2 Staging

L'infrastructure est entièrement hébergée sur une machine virtuelle Ubuntu 24.04.2 LTS installée sur le serveur w3 de la filière informatique. Le système s'articule autour de trois composants clés :

- **Le Client (Terminaux utilisateurs)** : Les utilisateurs (étudiants, professeurs et membres de la gouvernance) accèdent à l'application directement depuis le navigateur web de leur ordinateur de bureau (*desktop*). Le produit est conçu pour une utilisation exclusive sur ordinateur.
- **Le Serveur d'Application** : Une instance unique Ubuntu 24.04.2 LTS centralise le Front-end (Vue.js) et l'API Back-end (AdonisJS). C'est elle qui porte la logique métier, exécute l'algorithme de répartition sous contraintes et gère les états du système.
- **MySQL** : Une base de données MySQL assure la persistance et le stockage de toutes les données du projet (les comptes utilisateurs, les compétences des professeurs, les énoncés de projets, etc ...). **Pourquoi MySQL ?** J'avais besoin d'une base relationnelle solide pour m'assurer que des opérations sensibles, comme les votes ou les allocations, soient traitées sans aucune erreur d'intégrité. Au-delà de ces aspects techniques très robustes, je dois admettre que j'ai surtout privilégié MySQL parce que c'est la technologie que je maîtrise le mieux.

2.3.3 Développement

L'environnement de développement est conçu pour être simple à installer. Le frontend Vue.js et le backend AdonisJS tournent directement sur la machine de développement. Seule la base de données MySQL est conteneurisée via Docker, ce qui évite toute installation locale lourde tout en restant totalement transparente pour le reste du projet.

- **Prérequis** : Un développeur reprenant le projet doit installer deux outils sur sa machine : Node.js (version 20 ou supérieure) et Docker Desktop. Le développeur doit installer les dépendances une fois dans le projet. Aucune autre installation de base de données n'est nécessaire.
- **Base de données** : Un fichier docker-compose.yml situé à la racine permet de monter instantanément une instance MySQL. La base de données tourne de manière isolée et persiste ses informations grâce aux volumes Docker. Le développeur peut initialiser la structure des tables (compétences, projets, rôles) et injecter des utilisateurs de test (Gouvernance, Profs, Étudiants fictifs avec le seeder disponible) en une seule commande de migration et d'alimentation
- **Variables d'environnement** : Un fichier .env.example est présent dans le dépôt Github. Il contient la structure nécessaire au projet. Les données sensibles sont transmises de manière sécurisée entre les partis.

2.3.4 Technologies utiles :

Backend

- **AdonisJS v6** — Framework Node.js MVC pour construire des APIs REST. Gère le routing, les middleware, l'ORM et les migrations.
- **Lucid ORM** — ORM d'AdonisJS pour interagir avec la base de données via des modèles TypeScript.
- **MySQL** — Base de données relationnelle qui stocke les utilisateurs, projets, votes et répartitions.
- **TypeScript** — Langage typé qui compile vers JavaScript, utilisé pour le backend et le frontend.

Frontend

- **Vue 3** — Framework JavaScript réactif pour construire l'interface utilisateur avec des composants.
- **Pinia** — Store de gestion d'état pour Vue 3, utilisé pour l'authentification et les données globales.
- **Vue Router** — Gestion de la navigation entre les pages de l'application SPA.
- **Axios** — Client HTTP pour communiquer avec l'API backend.
- **Vite** — Outil de build et serveur de développement ultra-rapide pour le frontend.

Authentification

- **Microsoft Azure Active Directory** — Service d'identité Microsoft utilisé pour authentifier les utilisateurs via leurs comptes Eduvaud.

- **MSAL Browser** — Bibliothèque Microsoft pour gérer le flux des tokens

Infrastructure

- **Docker** — Conteneurisation de la base de données MySQL en développement.
- **PM2** — Gestionnaire de processus Node.js pour faire tourner le backend en production.

2.4 L'architecture et les services en schéma

2.4.1 Schéma de l'environnement de développement

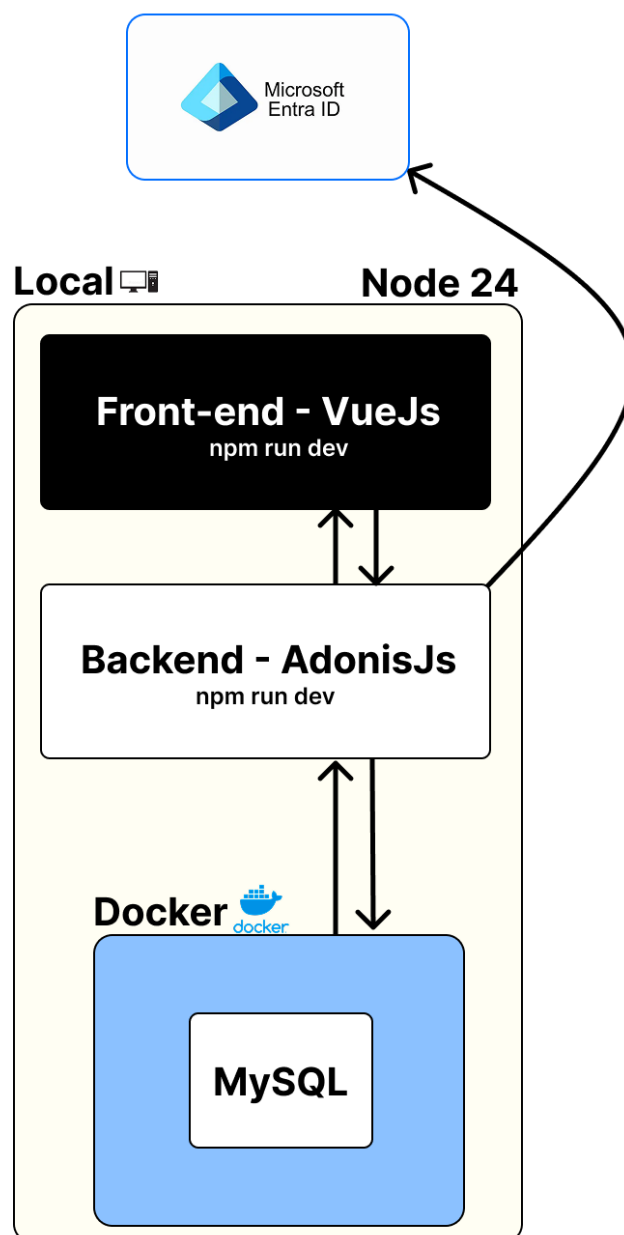


Figure 1 : Environnement de développement

2.4.2 Schéma de l'environnement de staging

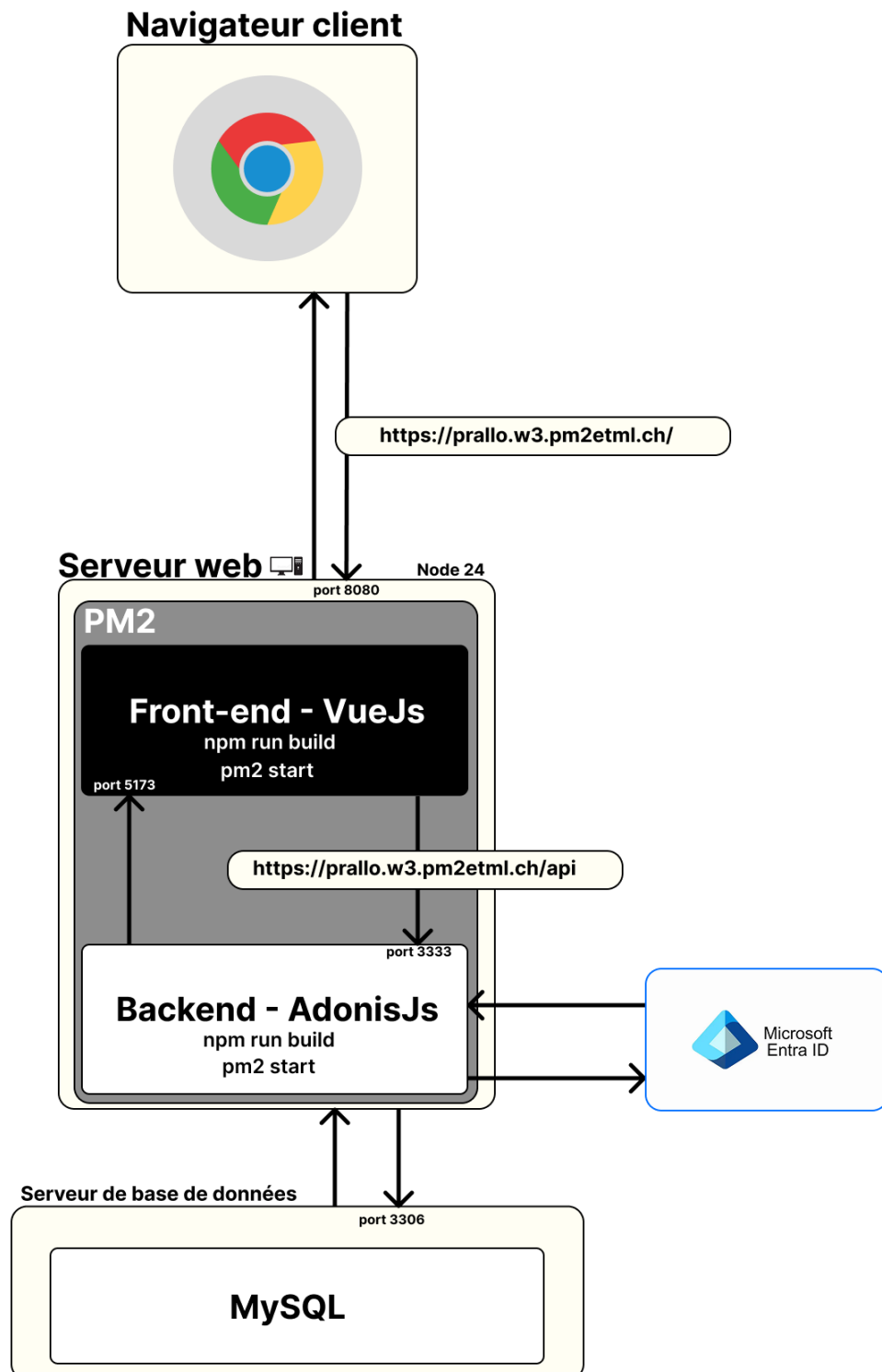


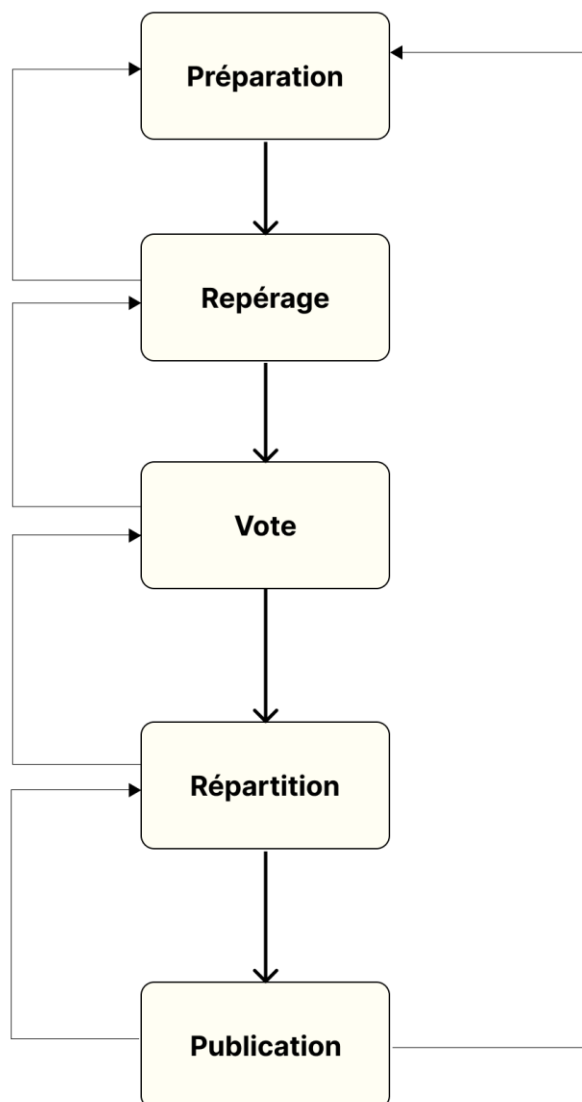
Figure 2 : environnement de staging

La disponibilité de l'application PRALLO est assurée par **PM2**. Cet outil sert de "gardien" pour le serveur web :

- **Gestion des processus** : Il fait tourner en arrière-plan le **Front-end (VueJS)** et le **Backend (AdonisJs)**.
- **Surveillance (Monitoring)** : Il permet également de surveiller en temps réel les performances du serveur, comme l'utilisation de la mémoire RAM et du processeur (CPU).

2.4.3 Digramme d'état

Etat du système PRALLO

*Figure 3 : Diagramme d'état*

Le diagramme ci-dessus illustre les différents états par lesquels passe le système PRALLO (**Préparation, Repérage, Vote, Répartition, Publication**).

Le flux de l'application est conçu de manière flexible : **il est possible à tout moment de passer à l'état suivant ou de revenir à l'état précédent** afin de s'adapter aux réalités du terrain et aux corrections nécessaires. Il n'est juste pas possible de revenir en arrière une fois que le cycle a été fini (c'est-à-dire, il est impossible de passer de « Publication » à « Préparation » et de revenir en arrière)

Cependant, changer d'état de manière non linéaire ou prématurée peut impacter la cohérence des données (par exemple, revenir à l'état de *Vote* alors qu'une *Répartition* a déjà été amorcée). C'est pourquoi **l'application affiche des messages d'avertissement (*warnings*)** lors de ces transitions. Ces alertes informent l'utilisateur des problèmes potentiels ou des pertes de données possible avant qu'il ne confirme son action, garantissant ainsi la flexibilité du système sans compromettre sa sécurité.

Voici un exemple d'utilisation des états de PRALLO :

1. **Préparation** : la période de P_APPRO approche et il faut donc lancer PRALLO, un administrateur initialise le site web. Comme convenu, la première connexion crée un compte doyen. La personne en charge communique à la gouvernance que le service est prêt et qu'ils doivent se connecter. L'administrateur leurs attribue ensuite le rôle de doyen. Les doyens demandent aux professeurs de se connecter. Il se passe exactement la même chose, mais avec les professeurs. La gouvernance communique une date à laquelle le site va passer en état « Repérage ». En attendant les professeurs choisissent leurs compétences et ajoutent des projets.
2. **Repérage** : Dans cet état, rien ne change pour les doyens et les professeurs. Les élèves/invités peuvent venir se renseigner sur les projets disponibles.
3. **Vote** : Les professeurs ne peuvent plus supprimer de projet, mais ils peuvent les modifier. Les élèves votent pour leurs projets préférés jusqu'à une date butoir.
4. **Répartition** : La répartition des projets est visible par tout le monde, seuls les doyens peuvent encore modifier les projets ou la répartition.
5. **Publication** : plus personne ne peut rien faire, hors-mi visualiser la répartition

Il est important de notifier que aucune condition pour passer d'un état à un autre n'est demandé

2.4.4 Diagramme de flux concernant l'authentification

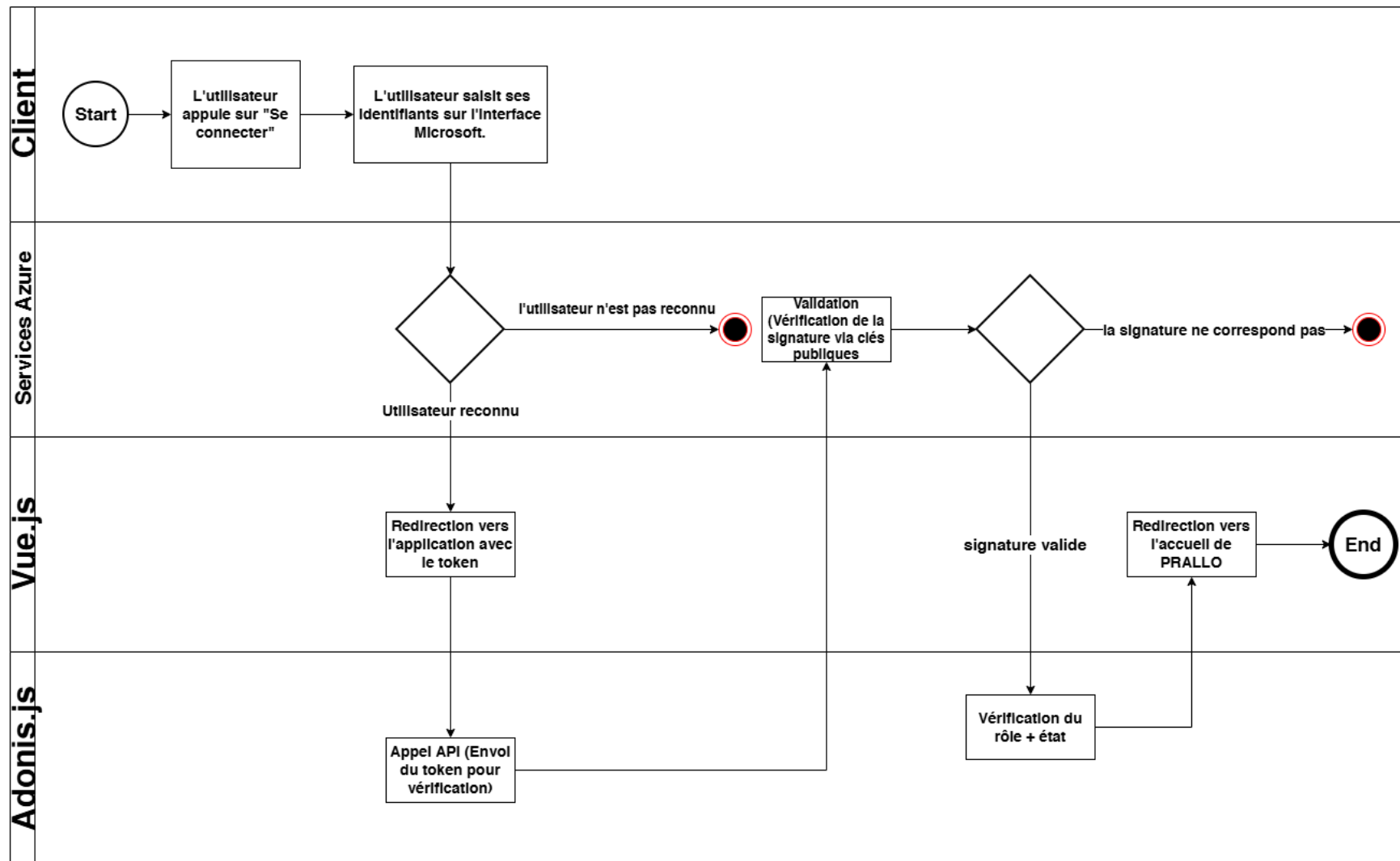


Figure 4 : diagramme de flux

Le diagramme d'activité ci-dessus détaille le processus d'authentification sécurisé et le routage dynamique mis en place au sein de l'application PRALLO. Ce flux s'appuie sur la bibliothèque **MSAL** (Microsoft Authentication Library) intégrée au Frontend et sur les **Services Azure** (Microsoft Entra ID) comme fournisseur d'identité.

Le processus se déroule selon les étapes suivantes :

1. **Phase d'authentification (Client & Azure)** : L'utilisateur initie la connexion depuis l'interface client. Il est alors invité à saisir ses identifiants directement sur l'interface sécurisée de Microsoft. Si l'utilisateur n'est pas reconnu par Azure, le processus s'interrompt immédiatement.
2. **Émission du Token (Azure & Vue.js)** : Une fois l'utilisateur authentifié avec succès, les Services Azure opèrent une redirection vers le Frontend **Vue.js** en lui transmettant un jeton d'accès sécurisé (Token).
3. **Vérification de sécurité (Adonis.js & Azure)** : Le Frontend transmet immédiatement ce jeton au Backend **Adonis.js** via un appel API d'authentification. Pour garantir l'intégrité du jeton, le Backend interroge les Services Azure afin de valider mathématiquement la signature via leurs clés publiques. Si la signature s'avère invalide ou altérée, la connexion est immédiatement rejetée.
4. **Contrôle d'accès et routage dynamique (Adonis.js & Vue.js)** : Dès que la signature est validée, le Backend exécute un middleware chargé de récupérer à la fois les permissions en fonction de son rôle et l'**état actuel du système PRALLO** (ex: Préparation, Vote, Répartition, Publication). C'est l'évaluation conjointe de ces deux variables (Rôle + État) qui dicte au routeur **Vue.js** vers quelle page d'accueil spécifique l'utilisateur doit être redirigé et quelles actions il est capable de faire.

2.5 Analyse fonctionnelle

2.5.1 Réalisation de maquettes

J'ai commencé par travailler sur figma pour réaliser des maquettes, une partie de celles-ci se trouvent en annexe. Cela m'a permis de mieux visualiser l'application et créer des questions pertinentes pour le client.

2.5.2 Discussion avec le Client/Product Owner

Lors de l'analyse du cahier des charges, il est clair que des points restent à éclaircir. C'est pourquoi dans la période d'analyse, j'ai pris du temps pour avoir des rendez-vous avec le product owner/client (Ici, le chef de projet, Monsieur Carrel, représente les 2 rôles en même temps).

Lors de ces discussions, j'ai posé plusieurs questions et **des changements dans le cahier des charges ont eu lieu** :

- **Authentification** : un point important de l'application PRALLO est l'authentification et l'attributions des rôles. Il était question de comment initialiser l'application, c'est-à-dire : Comme ce sont les doyens qui attribuent les rôles, comment fait-on pour créer le premier profil doyen ? passons-nous par la base donnée ? Avons-nous une liste de doyen que nous initialisons ? La réponse à cette question fut trouvée : **La première personne qui se connecte au service PRALLO, est enregistrée avec le rôle « Doyen »**, Grâce à ce système, l'administrateur qui initialisera le service, pourra changer les rôles des autres participants en doyen lors de leurs connexions.
- **Attribution des rôles** : Dans le paragraphe d'au-dessus, il est question de « changer le rôle des autres participants » mais c'est-à-dire ? Dans PRALLO, les doyens auront le droit d'attribuer des rôles au participants. **La gouvernance ne peut donc pas éditer librement la liste des participants, contrairement à ce que laisse penser le cahier des charges**. Elle ne peut pas supprimer de profils ni en créer, mais elle a le droit de modifier par exemple le rôle des invités
- **Rôle + Etat** : En ayant lu les paragraphes d'au-dessus, vous avez peut-être remarqué que nous parlons d'un rôle participant/invité et d'un état « Repérage ». Ceux-ci ne sont pas disponible dans le cahier des charges, **ce sont des ajouts au cahier des charges**, après discussion avec le Product Owner.

Les changements ne sont ici pas tous détaillés, les user story sont là pour définir les fonctionnalités disponibles dans l'application PRALLO.

2.5.3 User story

L'analyse fonctionnelle est constituée de toutes les user story réalisé dans le GitProject
Une User story est écrite de la manière suivante :

- **Un titre** : Il s'agit de la fonctionnalité que l'on souhaite développer.
- **Une description** : C'est ici que l'on indique qui veut pouvoir faire l'action, quelle action cette personne voudrais faire et pour quel but. Le formalisme est le suivant : *En tant que ... je veux ... afin de ...*
- **Les critères d'acceptation** : C'est ici que la User Story prend vie. En associant des maquettes visuelles à des règles de gestion précises, cette partie définit exactement le comportement de l'application. L'enjeu est d'être suffisamment minutieux pour que n'importe quel membre de l'équipe (développeur, testeur ou client) puisse se projeter dans l'application et en saisir immédiatement les moindres détails.

- **Gestion d'erreur** : Cette section liste les points de vigilance techniques et les erreurs potentielles à anticiper lors du développement (par exemple, la limite maximale de caractères pour un champ de saisie). Ces précisions sont indispensables, car elles servent de base directe à la conception des scénarios de test, conformément aux exigences de la Definition of Done (DoD) et de la stratégie de test globale du projet.
- **De labels** : L'application Prallo intégrant plusieurs rôles, il est essentiel de pouvoir s'y retrouver facilement dans le GitProject. Pour cela, l'utilisation des labels afin d'attribuer une étiquette précise à chaque tâche selon le profil concerné (par exemple : « Doyen » ou « Professeur »), permet de filtrer rapidement le tableau et de clarifier le suivi.

Les US se retrouvent en annexe.

2.6 Stratégie de test

Chaque niveau de test joue un rôle spécifique et indispensable pour s'assurer que le code livré répond aux attentes en termes de fiabilité, de performance et de convivialité.

1. Tests d'acceptance (Critères d'acceptance)

Les tests d'acceptance valident que le système répond exactement aux exigences métier définies dans les User Stories. Ils s'exécutent dans un contexte au plus proche de l'environnement de production et simulent un flux utilisateur complet.

- **Rôle** : Ils définissent clairement ce qu'un développement doit accomplir avant d'être considéré comme terminé. Un test réussi signifie que la fonctionnalité répond aux critères convenus avec les métiers.
- **Anticipation** : Ces cas de test sont documentés dans les User Story afin de valider les comportements attendus de bout en bout.

2. Tests unitaires (Jappa et Vitest)

Les tests unitaires permettent de vérifier la logique métier et les cas limites. Chaque composant est isolé et testé individuellement pour s'assurer qu'il gère correctement les erreurs.

Pour cette catégorie, le projet s'appuie sur l'outil suivant :

- **Jappa (API REST)** : Utilisé pour tester la logique du back-end, la validation des routes et la gestion des cas exceptionnels.

Il est convenu dans la DoD que les points sous la rubrique « Gestion d'erreur » font le point de test unitaire

3. Tests End-to-End (Cypress)

Les tests End-to-End (E2E) avec Cypress visent à vérifier l'intégration globale du système en simulant des scénarios réels, de l'authentification initiale jusqu'à la validation finale d'une action.

- **Validation visuelle et fonctionnelle** : Cypress parcourt l'interface comme un utilisateur réel, On s'assure qu'au moins un élément visuel clé s'affiche correctement sur chaque page du DOM.
- **Suivi et robustesse** : Ce framework garantit que l'expérience utilisateur reste cohérente, fluide et sans régression. De plus, il offre une excellente visibilité sur l'exécution des tests grâce à son interface interactive et ses replays vidéo.

Il est convenu dans la DoD qu'au minimum un élément visuel soit testé par vue.

2.7 Risques techniques

- **Disponibilité des systèmes externes** : L'application dépend entièrement de Microsoft Entra ID. Si le service Azure subit une panne ou une indisponibilité, le système d'authentification devient inaccessible et plus personne ne peut se connecter à PRALLO.
Solution : On pourrait imaginer un deuxième moyen de connexion si le premier tomberait en panne. Néanmoins ce projet ne contiendra qu'un moyen de connexion
- **Complexité de l'authentification MSAL** : Configurer les redirections entre le local, le staging et la prod est complexe.
Solution : être au clair avec le système MSAL et centraliser les données possibles dans le fichier .env
- **Complexité au niveau des fonctionnalités** : Vu l'exhaustivité des user story, Il existe un risque réel de passer à côté de certaines fonctionnalités secondaires ou de mal interpréter des cas limites lors du développement.
Solution : Utiliser un tableau de suivi (type Jira ou Trello) pour cocher rigoureusement chaque User Story.
- **Dépendance au résultat de l'intelligence artificielle pour rester dans les délais de livraison** : Comme la réalisation du code est fait entièrement avec l'aide de l'IA, si celui-ci prend beaucoup plus de temps qu'imaginé, que le code demande du débogage long, le délai de livraison pourrait en être impacté.
Solution : Planifier des code review à l'ajout de fonctionnalités importantes

2.8 Planification

Un tableau permet de visualiser la date à laquelle on aimerait avoir fini les fonctionnalités. Ces dates feront l'objet de « Sprint review » dans la partie 3. Réalisation. Si des retards ou des problèmes devraient avoir lieu, c'est plus tard que le rapport en discutera.

Titre de l'user story	Rôle impliqué	Date prévu
S'authentifier	Utilisateur	26.05.26
1ère authentification	Administrateur	26.05.26
Gérer l'état du système	Doyen	26.05.26
Afficher la liste des participants	Doyen	26.05.26
Visualiser les projets	Doyen	26.05.26
Visualiser les projets	Professeur	26.05.26
Visualiser les projets	Élève	26.05.26
Visualiser les projets	Invité	26.05.26
Suppression de projet	Doyen	26.05.26
Modifier des projets	Doyen	26.05.26
Visualiser mes compétences	Professeur	26.05.26
Visualiser les compétences de PRALLO	Professeur	26.05.26
Ajouter des compétences	Professeur	26.05.26
Supprimer des compétences de ma liste	Professeur	26.05.26
Changer le statut des participants	Doyen	26.05.26
Attribuer un nombre de projet	Doyen	26.05.26
Ajouter des projets	Professeur	28.05.26
Supprimer mes projets	Professeur	28.05.26
Modifier mes projets	Professeur	28.05.26
Voter pour un projet	Élève	28.05.26
Supprimer mon vote d'un projet	Élève	28.05.26
Visualiser les projets votés	Élève	28.05.26
Mettre dans l'ordre de préférences	Élève	28.05.26
Visualiser les compétences	Doyen	28.05.26
Supprimer les compétences	Doyen	28.05.26
Fusionner des compétences	Doyen	28.05.26
Ajouter des compétences à ma liste	Professeur	28.05.26
Faire la répartition	Gouvernance	28.05.26

3 Réalisation

3.1 Déroulement

La réalisation s'est déroulée en 4 étapes, celles-ci seront détaillées dans les paragraphes suivants :

3.1.1 Prompt Bullet

Le projet PRALLO était pour moi l'occasion idéale de tester les limites de l'IA dans un contexte de développement réel. L'idée n'était pas seulement de "générer du code", mais de voir comment un étudiant peut réellement transformer son processus analytique en vrai code utilisable.

Pour relever ce défi, j'ai mis au point une méthode que j'ai appelée « **Prompt Bullet** ». Plutôt que de solliciter l'IA de manière isolée, j'ai tout misé sur la préparation en amont, c'est pourquoi il y a eu tant d'analyse dans le projet :

- **Regrouper les fichiers** : J'ai commencé par regrouper toute ma documentation dans un dossier « Annexe ». C'est-à-dire qu'il contenait mes user story au format word + md, mes maquettes et mon rapport avec la partie analyse de faite, il contenait donc déjà des schéma et des explications des environnements/architectures de PRALLO
- **La création d'un prompt** : Au lieu de poser des questions de manière itérative, par exemple, « génère moi la liste des rôles avec les permissions », je lui ai directement demandé de tout faire. Pour se faire, j'ai créé un fichier md qui regroupait toutes mes demandes et pour lui dire quels fichiers analyser.
- **L'IA comme accélérateur, l'humain comme pilote** : En donnant à l'IA ce prompt, il m'a généré une base de code très solide. La grande majorité des logiques métiers étaient présente lors de la première itération. C'était assez impressionnant à voir car il avait vite compris les concepts de l'application, notamment par exemple autour de états de PRALLO par exemple. Néanmoins, Claude n'a pas su respecter mes maquettes, le front-end ne ressemblait pas du tout à ce que j'avais fait.

Au final, cette approche m'a permis d'être beaucoup plus efficace. J'ai gardé le rôle de chef d'orchestre, m'assurant que chaque bloc généré respectait scrupuleusement les spécifications du projet tout en me laissant la liberté de peaufiner les détails cruciaux.

3.1.2 MCD et MLD

Une fois le code avec les migrations réalisé, on peut s'attaquer à corriger/schématiser la base de donnée.

Le **MCD** suivant est donc crée :

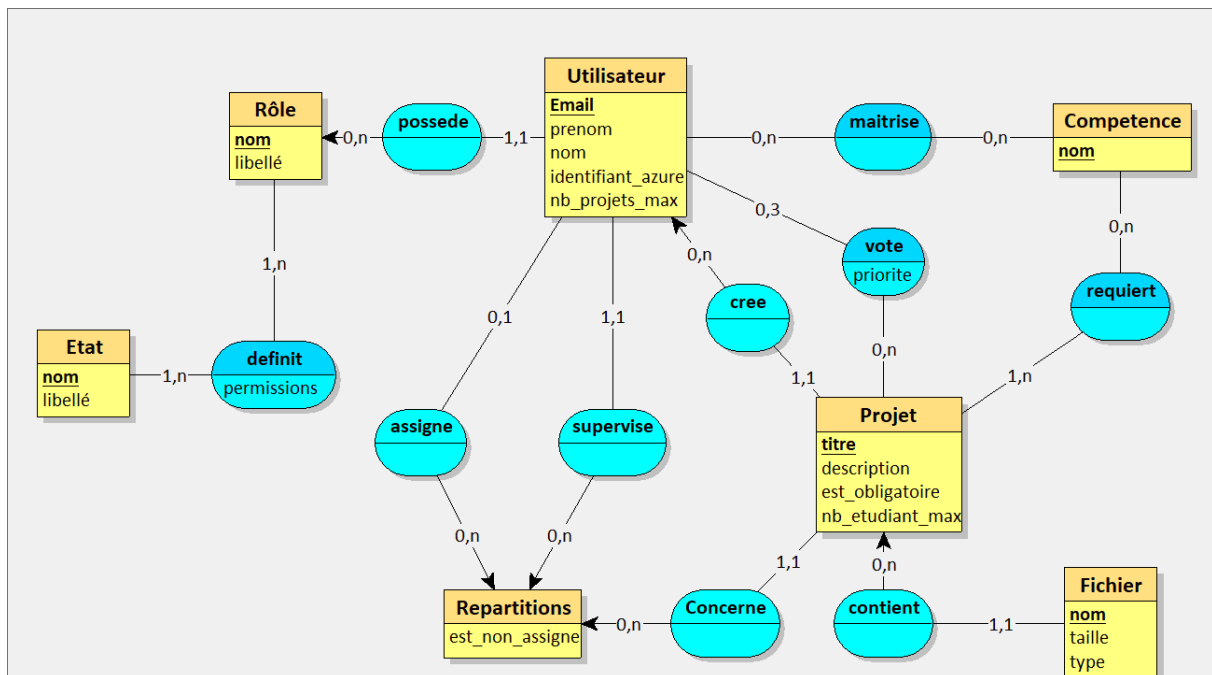


Figure 5: MCD

Le MCD de PRALLO modélise les données nécessaires à la gestion du processus d'attribution de projets de fin d'année. Il représente sept entités principales dont les identifiants naturels sont soulignés et en gras dans le diagramme. Les identifiants utilisés dans ce MCD sont tous des identifiants naturels, c'est-à-dire des attributs métier qui existent indépendamment de toute implémentation technique. Aucune clé artificielle (type auto-incrément) n'apparaît dans le modèle conceptuel. L'adresse email identifie un utilisateur, le nom identifie un rôle, un état ou une compétence, le titre identifie un projet, et la combinaison nom + projet identifie un fichier.

- **Choix de modélisation notables :**

1. **Utilisateur** joue plusieurs rôles fonctionnels : une seule entité représente à la fois les professeurs, les étudiants, les doyens et les invités. C'est le rôle associé qui détermine le comportement de l'utilisateur — un professeur crée des projets et supervise des répartitions, un étudiant vote et est assigné à un projet.
2. **Vote** est une association entre Utilisateur et Projet portant l'attribut priorité. Un étudiant peut exprimer au maximum 3 votes classés (1er, 2ème, 3ème choix), d'où la cardinalité (0,3) côté Utilisateur. Un projet peut recevoir autant de votes qu'il y a d'étudiants.

3. **Répartition** est modélisée comme une entité car elle implique trois participants , un projet, un étudiant et un professeur, ce qui correspond à une association ternaire que Looping ne supporte pas nativement. Elle porte l'attribut `est_non_assigné` qui distingue une place volontairement vide (projet obligatoire sans étudiant disponible) d'une place non encore traitée.
4. **Définit** est une association entre Rôle et État portant l'attribut `permissions`. Elle encode les 22 droits applicables pour chaque combinaison rôle/état, permettant de contrôler finement ce que chaque rôle peut faire à chaque étape du processus sans modifier le code applicatif.

Le MLD qui en découle est donc le suivant :

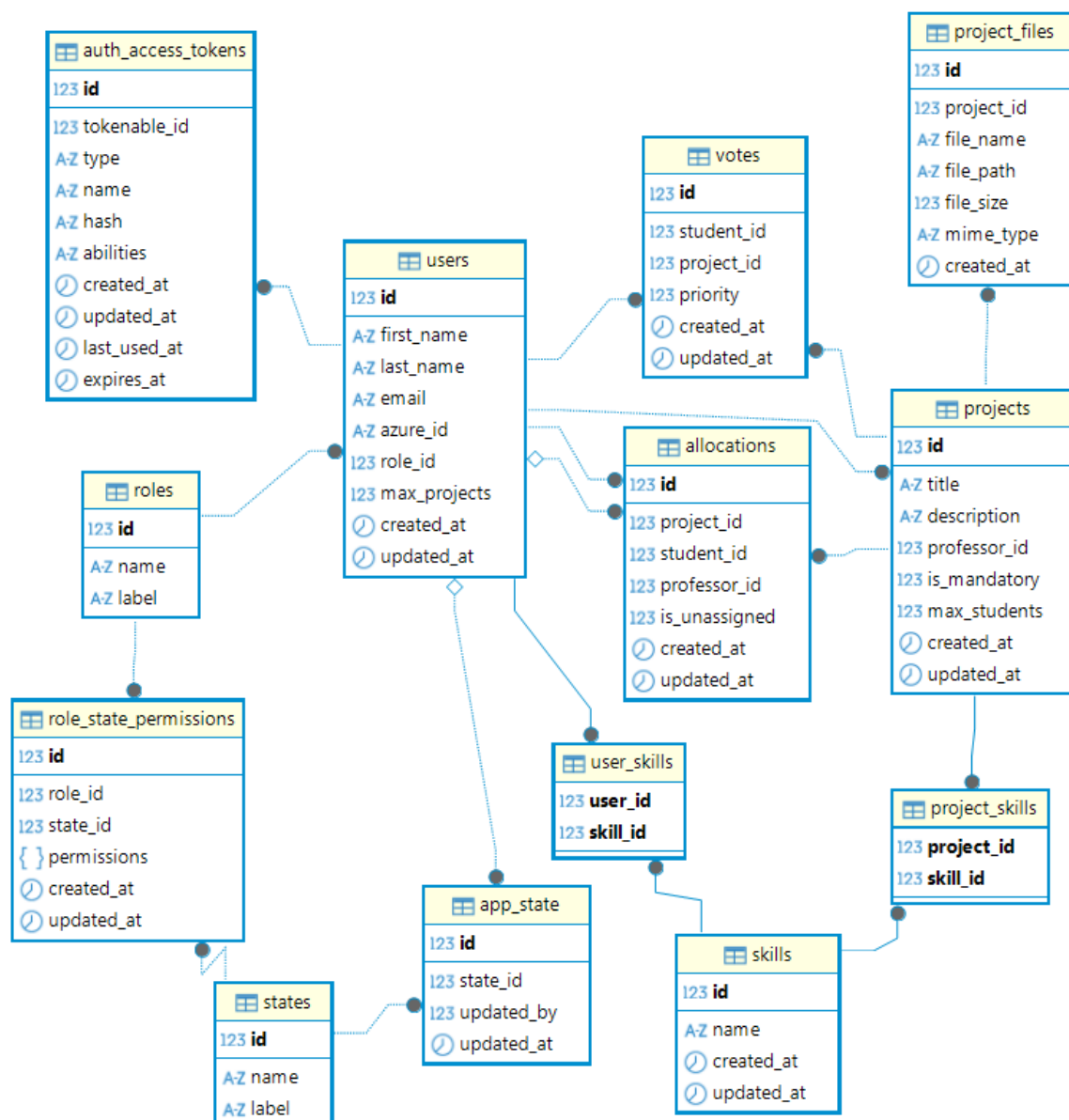


Figure 6 : MLD

Pour la gestion de la base de données, j'ai opté pour l'outil **DBeaver** plutôt que l'interface en ligne de commande de Docker. Ce choix a été essentiel pour gagner en efficacité, car l'interface graphique de DBeaver m'a permis de visualiser clairement la structure des tables, d'inspecter directement les champs et de vérifier les relations entre les données en un coup d'œil, rendant mes manipulations bien plus simples qu'avec les commandes textuelles. C'est de cet outil qu'est tiré la capture d'écran visible au-dessus.

Concernant le MLD : la gestion des rôles/permissions en base de données a été discuté avec M. Carrel. Le choix de stocké des permissions au format JSON dans la base de données a été pris.

3.1.3 Feature/Bugfix

Une fois la première étape terminée, je me suis attaqué aux fonctionnalités à ajouter ou aux bugs à corriger. Pour chaque feature ou bugfix, j'ai créé une branche depuis la branche develop. On a donc le suivie des fix/feature disponible sur github. Ici je vais détailler quelques fonctionnalités de prallo importantes pour ne pas toutes les citer :

- **Rôles et permissions** : L'application implémente un système de contrôle d'accès basé sur les rôles (RBAC — Role-Based Access Control). Quatre rôles sont définis : doyen, professeur, étudiant et invité. Les permissions ne sont pas fixes par rôle, mais varient également selon l'état actuel de l'application (vote, répartition, publication, par exemple), ce qui permet de contrôler finement ce que chaque rôle peut faire à chaque étape du processus. Cette combinaison rôle × état est stockée en base de données sous forme d'un objet JSON contenant toutes les permissions distinctes. À chaque requête, un premier middleware charge les permissions de l'utilisateur connecté pour l'état courant, puis un second vérifie que la permission requise pour l'action demandée est accordée. Ce second middleware supporte une logique OU, permettant à une même route d'être accessible si au moins l'une des permissions requises est présente, sans avoir à dupliquer la route.
- **Authentification** : L'authentification repose sur Microsoft Azure Voici comment cela fonctionne concrètement :
 1. **Un accès simplifié et sécurisé** : Lorsqu'un utilisateur souhaite se connecter, l'application le redirige vers la page officielle de Microsoft. Une fois identifié, Microsoft nous renvoie un jeton de sécurité (un « token ») qui sert de preuve d'identité pour la session.

2. **Vérification automatique** : À chaque action dans l'application, ce jeton est transmis de manière invisible. Le système vérifie systématiquement la signature numérique du jeton auprès de Microsoft pour s'assurer qu'il est authentique. Cela garantit une sécurité totale, car il est impossible de falsifier ces identifiants.
3. **Création automatique des profils** : Lors de la première connexion, le système identifie automatiquement l'utilisateur et lui attribue un rôle. Par défaut, tout nouvel utilisateur est créé avec le rôle « invité ». Toutefois, pour assurer le démarrage du projet, le tout premier utilisateur enregistré est automatiquement promu au rang de « doyen » pour garantir l'administration initiale du système.
4. **Une approche moderne (Stateless)** : L'un des grands avantages de ce système est qu'il est entièrement autonome (« stateless »). L'application n'a pas besoin de stocker de fichiers de session ou de cookies sur le serveur, ce qui rend le système beaucoup plus léger, rapide et moins sensible aux failles de sécurité classiques.

Comme l'authentification est « Géré » par Microsoft, **aucun mot de passe n'est stocké en base de données**, on ne doit donc pas s'attarder au hashage et à la sécurité des données comme **aucune données sensibles n'est stocké**.

- **Fonction de répartition** : La répartition automatique des étudiants aux projets est calculée en trois phases. La phase 1 utilise l'algorithme hongrois, un algorithme d'optimisation combinatoire qui minimise le coût global d'affectation selon les votes exprimés. Une matrice de coûts est construite en attribuant un coût croissant selon la priorité du vote (1er choix, 2ème, 3ème), les projets pouvant accueillir plusieurs étudiants étant expansés en autant de slots distincts. L'algorithme produit l'affectation globalement optimale selon les préférences. La phase 2 traite les projets obligatoires non encore complets en y assignant de force les étudiants non affectés. La phase 3 distribue les étudiants encore libres sur les projets optionnels vides. Dans les trois phases, l'attribution d'un professeur suit une logique greedy : parmi les professeurs ayant une compétence correspondante au projet et une capacité disponible, celui avec la charge actuelle la plus faible est sélectionné en priorité, équilibrant naturellement la répartition du travail.

3.1.4 Tests

Contrairement à la méthode du Test-Driven Development où l'on rédige les tests en amont, j'ai fait le choix personnel de développer mon code avant de passer à la phase de test. J'ai ensuite utilisé l'intelligence artificielle comme un véritable assistant pour m'aider à rédiger mes tests. Mais attention, je n'ai jamais fait confiance aveuglément à ce qu'elle produisait : j'ai pris le temps de relire ce que Claude m'avait fourni. Pour moi, les tests sont importants, et je sais qu'un test mal écrit est un piège dangereux qui peut donner une fausse impression de sécurité. L'IA, aussi performante soit-elle, pourrait très bien générer des tests qui semblent parfaits en surface mais qui ne vérifient rien du tout, voire qui valident des comportements incorrects. C'est pourquoi j'ai analysé le plus de test possible dans le temps imparti pour m'assurer que les conditions, les données et les résultats attendus collaient parfaitement à la réalité de mon code.

3.1.5 Mise en Staging

L'objectif de cette étape était de déployer l'application sur un environnement de staging accessible publiquement. Étant donné que le domaine et l'infrastructure n'existaient pas initialement, l'ensemble a dû être configuré depuis zéro avec l'appui de Monsieur Melly. L'architecture retenue repose sur un serveur HTTP statique pour le front-end et un serveur Node.js pour le back-end.

Le déploiement s'est effectué selon les étapes suivantes :

1. **Connexion distante** : La connexion au serveur a été établie via SSH avec la commande `ssh user@<adresse-du-serveur>`.
2. **Importation des builds** : Les artefacts de production ont été importés via le gestionnaire de services de l'ETML, après avoir été générés localement avec les commandes suivantes :
 - `node ace build` pour le back-end.
 - `npm run build` (avec l'option `only` pour ignorer les erreurs TypeScript) pour le front-end.
 - Les fichiers `.env` ont été préalablement adaptés à l'environnement de staging.
3. **Installation de PM2** : Le gestionnaire de processus **PM2** a été installé globalement sur le serveur back-end à l'aide de la commande `npm install -g pm2`.
4. **Lancement du service** : Le serveur back-end a été démarré via PM2 avec la commande `pm2 start ace.js --name PRALLO_API`.

Le suivi des services s'effectue directement sur le serveur via la commande `pm2 list`, permettant de visualiser les processus actifs, comme illustré dans la capture d'écran suivante :

```

p167pqq@INF-B22-M203 MINGW64 ~
$ ssh -p 222 pm2etml-api-prallo-w3@api.prallo.w3.pm2etml.ch
pm2etml-api-prallo-w3@api.prallo.w3.pm2etml.ch's password:

#####
##                               ##
#####

* Website:      https://www.cloudpanel.io
* Documentation: https://www.cloudpanel.io/docs/v2/
* Best Practices: https://www.cloudpanel.io/docs/v2/best-practices/
* CloudPanel:   https://admin.w3.pm2etml.ch/
* CloudPanel CLI: clpctl

Last login: Thu May 28 15:36:46 2026 from 193.5.248.23
pm2etml-api-prallo-w3@w3:~$ pm2 list


```

id	name	mode	u	status	cpu	memory
0	PRALLO_API	fork	3	online	0%	93.8mb

```

pm2etml-api-prallo-w3@w3:~$ pm2 show PRALLO_API
Describing process with id 0 - name PRALLO_API

```

Figure 7 : SSH avec pm2

Néanmoins, plusieurs problèmes se sont enchaînés. Le backend ne démarrait pas à cause d'une dépendance manquante, résolue en relançant npm install directement sur le serveur. La configuration CORS bloquait toute communication entre le frontend et l'API, nécessitant l'ajout du domaine de staging dans config/cors.ts suivi d'un rebuild. Après ces retouches la redirections ne fonctionnent toujours pas. Il faudrait investir un peu plus de temps dans l'environnement de staging pour qu'il fonctionne.

3.2 Sprints

3.2.1 Fin du premier Sprint, 26.05.2026

A la fin de journée 26.05, si on regarde le tableau suivant :

S'authentifier	Utilisateur	26.05.26
1ère authentification	Administrateur	26.05.26
Gérer l'état du système	Doyen	26.05.26
Afficher la liste des participants	Doyen	26.05.26
Visualiser les projets	Doyen	26.05.26
Visualiser les projets	Professeur	26.05.26
Visualiser les projets	Élève	26.05.26
Visualiser les projets	Invité	26.05.26

Suppression de projet	Doyen	26.05.26
Modifier des projets	Doyen	26.05.26
Visualiser mes compétences	Professeur	26.05.26
Visualiser les compétences de PRALLO	Professeur	26.05.26
Ajouter des compétences	Professeur	26.05.26
Supprimer des compétences de ma liste	Professeur	26.05.26
Changer le statut des participants	Doyen	26.05.26
Attribuer un nombre de projet	Doyen	26.05.26

Toutes les fonctionnalités hors-mi l'authentification sont « presque » finis et disponibles dans le kanban git Project sous la rebrique in-review. C'est à ce moment là qu'entre en jeu ma Définition of Done. Si la user story se trouve dans la rebrique in-review, elle doit passer les critères de la DoD pour finir en « Done ». Pai ailleurs ma DoD est la suivante :

- Si la tâche contient des commentaire sous # Gestion d'erreur, celle(s)-ci doivent être contrôlé par des tests unitaires/fonctionnelles
- La US a été validé par le Product Owner
- Le front-end a été validée par l'équipe de développement
- Au moins 1 élément visuel a été testé via un test E2E

Elle est aussi disponible dans la description du gitProject.

Durant ce sprint, j'ai :

- **Organisation et Workflow** : J'ai structuré le travail autour du modèle **Gitflow** pour garder une base de code propre et organisée. Aujourd'hui, tout ce qui a été développé est centralisé dans le Kanban de GitProject, dans la colonne *In Review*. En gros, le gros du travail est fait, il ne reste plus qu'à passer chaque fonctionnalité au crible de notre *Definition of Done* pour les valider officiellement.
- **Méthodologie "Prompt Bullet"** : Pour gagner en efficacité, j'ai testé une approche itérative en m'appuyant sur l'IA. J'ai conçu un "prompt maître" au format Markdown qui intégrait toute notre documentation technique, ce qui m'a permis de générer une première version solide des fonctionnalités très rapidement. Bien sûr, cette base "full IA" n'était qu'une étape : j'ai ensuite repris chaque partie pour effectuer une revue technique détaillée, m'assurer que le code était cohérent et qu'il correspondait exactement à nos besoins.
- **Fiabilisation et Correctifs** : Une fois la structure en place, j'ai passé pas mal de temps à "nettoyer" et ajuster le résultat. J'ai corrigé les bugs qui apparaissaient lors de mes tests et j'ai ajouté les fonctionnalités qui manquaient ou qui méritaient quelques finitions sur la logique métier. L'objectif était vraiment de transformer cette base générée en un produit qui correspond au cahier des charges

3.2.2 Fin du deuxième sprint, 28.05.2026

A la fin de journée 28.05, si on regarde le tableau suivant :

S'authentifier	Utilisateur	26.05.26
1ère authentification	Administrateur	26.05.26
Ajouter des projets	Professeur	28.05.26
Supprimer mes projets	Professeur	28.05.26
Modifier mes projets	Professeur	28.05.26
Voter pour un projet	Élève	28.05.26
Supprimer mon vote d'un projet	Élève	28.05.26
Visualiser les projets votés	Élève	28.05.26
Mettre dans l'ordre de préférences	Élève	28.05.26
Visualiser les compétences	Doyen	28.05.26
Supprimer les compétences	Doyen	28.05.26
Fusionner des compétences	Doyen	28.05.26
Ajouter des compétences à ma liste	Professeur	28.05.26
Faire la répartition	Gouvernance	28.05.26

Toutes les fonctionnalités fonctionnent, je suis donc dans les temps en termes de user story. Néanmoins ils manquent les tests et l'environnement de staging.

Durant ce sprint, j'ai :

- **Gitflow** : Travaillé sur le code en respectant le workflow Gitflow, en ajoutant et affinant les fonctionnalités de PRALLO sur des branches dédiées.
- **Staging** : Commencé à mettre en place l'environnement de staging. Le domaine n'existait pas initialement et rien n'était configuré, j'ai donc dû tout initialiser avec l'aide de M. Melly. J'ai ensuite déployé le code sur les serveurs, mais l'environnement présente encore des problèmes : les modifications ne semblent pas être prises en compte correctement par le serveur.

3.3 Description des tests effectués

Deux niveaux de tests ont été mis en place pour valider le comportement de l'application.

Les tests fonctionnels backend ont été réalisés avec le framework Japa, intégré à AdonisJS. Ils couvrent la gestion d'erreurs sur les compétences, les projets, les votes, les permissions par rôle et l'authentification. Chaque groupe de tests configure préalablement l'état de l'application via le modèle AppState directement en base, afin de garantir que les permissions correspondent à l'action testée. Par exemple, les tests de validation des projets s'exécutent en état préparation (où les professeurs ont le droit de créer), tandis que les tests de votes s'exécutent en état vote. L'authentification utilise le mécanisme de bypass dev activé en environnement non-production. Les tests se lancent avec node ace test.

Les tests E2E frontend ont été réalisés avec Cypress. Ils couvrent les vues login, projets, compétences, votes, participants, répartition et gestion d'état, soit 7 fichiers de test.

Les tests couvrent :

- **Tests fonctionnels backend (Japa) — 20 tests**

Compétences : rejet d'une compétence en doublon, rejet d'un nom vide, rejet de la fusion d'une compétence avec elle-même.

Projets : rejet d'un titre vide, rejet d'un titre supérieur à 30 caractères, rejet d'une description vide, rejet d'une description supérieure à 350 caractères, rejet d'un tableau de compétences vide, rejet de plus de 20 compétences, rejet d'un titre en doublon, interdiction pour un professeur de modifier le projet d'un autre.

Votes : rejet d'un 4ème vote, rejet d'un vote en doublon pour le même projet.

Permissions : un étudiant ne peut pas créer de projet, un non-doyen ne peut pas changer l'état, un non-doyen ne peut pas supprimer une compétence, rejet d'une requête sans token.

Authentification : rejet d'une requête sans token, rejet d'un token invalide, rejet d'un email dev inexistant.

- **Tests E2E frontend (Cypress) — 7 fichiers**

Login : affichage du logo et du bouton de connexion, connexion via bypass dev avec différents rôles, redirection après connexion, affichage d'une erreur en cas d'échec.

Projets : affichage de la liste, affichage des détails, ouverture et fermeture du modal, recherche par texte, bouton de création visible pour le doyen, validation du titre et de la description.

Compétences : affichage de la liste, ajout d'une compétence, rejet d'un doublon, suppression avec confirmation, fusion de deux compétences.

Votes : affichage des projets en état vote, affichage des votes de l'étudiant, suppression d'un vote.

Participants : affichage du tableau, affichage des informations par participant, modification du rôle, affichage du champ nb projets max pour les professeurs.

Répartition : affichage de la vue, affichage des lignes après passage en état répartition, lecture seule pour les étudiants.

État : affichage de l'état actuel, boutons de transition, confirmation avant changement, annulation du changement.

Les captures d'écrans suivantes montrent les résultats des tests backend, et l'interface de cypress.

```
[ info ] booting application to run tests...

functional / Compétences – gestion d'erreurs (tests\functional\prallo_errors.spec.ts)
  ✓ rejette une compétence en double (case-insensitive) (284.59ms)
  ✓ rejette une compétence avec nom vide (14.65ms)
  ✓ rejette la fusion d'une compétence avec elle-même (23.45ms)

functional / Projets – gestion d'erreurs (tests\functional\prallo_errors.spec.ts)
  ✓ rejette un titre vide (18.99ms)
  ✓ rejette un titre supérieur à 30 caractères (9.63ms)
  ✓ rejette une description vide (9.37ms)
  ✓ rejette une description supérieure à 350 caractères (8.42ms)
  ✓ rejette un tableau de technologies vide (10.19ms)
  ✓ rejette plus de 20 technologies (8.88ms)
  ✓ rejette un titre en double (case-insensitive) (35.34ms)
  ✓ interdit à un professeur de modifier le projet d'un autre (30.7ms)

functional / Votes – gestion d'erreurs (tests\functional\prallo_errors.spec.ts)
  ✓ rejette un 4ème vote (max 3) (63.23ms)
  ✗ rejette un vote en double pour le même projet (22.63ms)

functional / Permissions par rôle – erreurs (tests\functional\prallo_errors.spec.ts)
  ✓ un étudiant ne peut pas créer de projet (9.29ms)
  ✓ un non-doyen ne peut pas changer l'état (6.81ms)
  ✓ un non-doyen ne peut pas supprimer une compétence (15.71ms)
  ✓ une requête sans token est rejetée avec 401 (1.51ms)

functional / Auth – gestion d'erreurs (tests\functional\prallo_errors.spec.ts)
  ✓ rejette une requête sans token (1.59ms)
  ✓ rejette un token invalide (2.52ms)
  ✓ rejette un email dev inexistant (3.01ms)
```

Figure 8 : tests backend

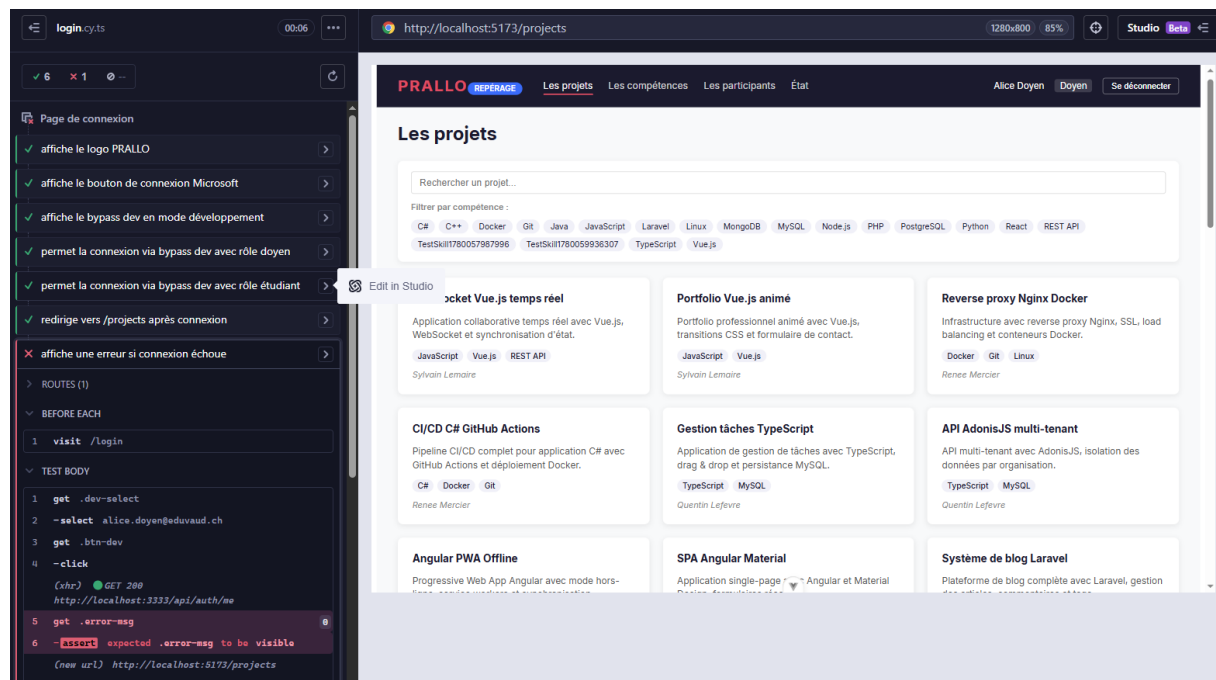


Figure 9 : tests avec cypress

```

Votes – gestion d'erreurs / rejette un vote en double pour le même projet

Expected - 1
Received + 1

409
400

AssertionError: expected 400 to equal 409

- at Assert.equal (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/assert/build/index.js:14:14)
- at Object.executor (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/tests/functional/prallo_errors.js:158:14)

153 |         if (!Array.isArray(votes) || votes.length === 0) return
154 |         const existingVote = votes[0]
155 |         // Réutiliser la même priorité pour éviter le conflit de priorité
156 |         const res = await client.post('/api/votes').header('Authorization', token)
157 |         .json({ projectId: existingVote.project.id, priority: existingVote.priority })
158 |         assert.equal(res.status(), 409)
159 |         assert.equal(res.body().error, 'ALREADY_VOTED')
160 |     })
161 | })
162 |
163 | // -----

- at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
- at async #wrapTestInTimeout (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async TestRunner.run (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async Test2.exec (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async GroupRunner.run (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async Group.exec (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async SuiteRunner.run (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)
- at async Suite.exec (C:/Users/pl67pqq/Desktop/TPI/src/backend/PRALLO_api/node_modules/@japa/core/build/index.js:14:14)

```

Figure 10 : détail d'une erreur sur les tests backend

On peut par ailleurs voir des erreurs sur les captures d'écrans. L'erreur du backend est dû au fait que l'on test les votes d'un élève, mais l'élève est initialisé dans la base de données avec des votes déjà présent. De ce fait, il est impossible de tester le vote double simplement, il faudrait supprimer un vote de la base ou utiliser un autre élève sans vote par exemple.

Les erreurs cypress sont souvent relié à l'état de l'application (AppState), en effet, des fois des vu sont bloqué (pas la permissions) en fonction de l'état. De ce fait, si le test n'utilise pas le bon Appstate, on n'a même pas accès la vue.

3.4 Bilan

3.4.1 Erreurs restantes

À la date du 31 mai 2026, les points bloquants identifiés sur l'environnement de staging sont les suivants :

- **Problème de configuration CORS** : La communication entre le front-end et l'API reste instable sur l'environnement de staging en raison de restrictions CORS non résolues. Bien que le site soit accessible en ligne, ces blocages empêchent le bon fonctionnement des fonctionnalités principales de l'application.

- **Problème de Typescript** : Concernant le build du front-end, je suis obligé d'utiliser l'option --only pour passer outre certaines erreurs TypeScript. Pour être clair : ce ne sont que des soucis de typage qui n'ont aucune incidence sur le fonctionnement concret du site. C'est juste une question de configuration technique que je n'ai pas encore eu le temps de finaliser.

3.4.2 Sprints

Voici le résultat de mon journal de travail :

Total heures TPI			90 heures						
Plannification Effective						Plannification Initiale			
Type d'activité	Total heures	en %				Type d'activité	Total heures	en %	
Analyse	34 h 25 min	37.01%				Analyse	27 h 00 m	30.00%	
Implémentation	24 h 55 min	26.79%				Implémentation	27 h 00 m	30.00%	
Test	2 h 15 min	2.42%				Test	9 h 00 m	10.00%	
Documentation	31 h 05 min	33.42%				Documentation	27 h 00 m	30.00%	
Absent	0 h 20 min	0.36%				Absent	0 h 00 m	0.00%	
Total	93 h 00 min	103.33%				Total		#####	

Figure 11: planification

On voit que l'analyse a pris plus de temps que prévu, de ce fait l'implémentation a pris moins de temps. On remarque aussi que les tests ne m'ont pas du tout pris trop de temps, grâce à l'utilisation de l'IA.

On notifie aussi que j'ai réalisé 3 heures supplémentaire pour la réalisation de ce TPI

Si on revient sur ma planification suivante :

#	Objectifs	Nombres d'heures	Revue
1	Introduction + introduction du Rapport	6	Vendredi 08.05
2	Réalisation des maquettes + User Story (Analyse)	25	Lundi 18.05
3	Ecriture de la partie 2. Analyse / Conception du Rapport	13	Mercredi 20.05
4	Réalisation du code + Partie 3. Réalisation du Rapport	28	Mercredi 27.05
5	Réalisation des tests E2E + Fonctionnels	5	Mercredi 27.05
6	Mise en Production	10	Vendredi 29.05
7	Conclusion	3	Lundi 01.06
	Total	90	

Je n'ai pas eu de retard concernant cette planification

Concernant les user story, celle-ci ont déjà été discuté

3.4.3 Dette technique

- **Logique métier**

En travaillant avec l'IA, j'ai eu tendance à laisser la logique métier s'accumuler directement dans les contrôleurs, car c'était le moyen le plus simple et le plus rapide pour obtenir un résultat fonctionnel via l'IA. Bien que cela m'ait permis d'avancer efficacement, je suis conscient que cette accumulation rend le code moins bien structuré et plus difficile à maintenir sur le long terme.

- **Commentaires**

Certes, l'IA ajoute des commentaires quand elle génère du code qu'elle juge complexe ou "difficile", mais cela ne suffit pas toujours à rendre le tout parfaitement clair.

- **Atomicité des commits**

Enfin, ma manière de travailler avec Git a été impactée par ce rythme soutenu de développement assisté par l'IA. À force de générer des blocs de fonctionnalités complets pour avancer rapidement, j'ai eu du mal à maintenir des commits atomiques. En fait quand je lui demandais de changer quelques choses, il avait l'habitude de toucher à plusieurs fichiers, méthode etc... Cela n'est pas du tout bon d'un point de vue gitflow.

La solution aurait été de mieux configurer l'environnement de développement mais cette partie sera décrite dans la suite du rapport

3.5 Recours à l'intelligence artificielle

Pour ce projet, l'utilisation de l'intelligence artificielle était nettement encouragée. Alors, je me suis posé les questions suivantes : qu'est-ce qu'on peut vraiment accomplir en 90 heures ? Est-ce qu'on apprend réellement en les utilisant ? L'utilisation de l'IA a ses pours et ses contres, ils seront détaillés plus tard. Quand je parle d'IA, je parle de :

- **Claude Code** : j'ai utilisé claude code pour la réalisation du code. La version utilisée fut la version payante fourni grâce à une licence de mon chef de projet. J'ai décidé d'utiliser claude code car j'ai pu directement l'utiliser dans l'IDE et que le contexte était disponible pour toute l'application. De plus, après avoir utilisé plusieurs IA lors de ma formation, j'ai trouvé que celle-ci comprenait mieux le contexte des applications que je voulais construire et produisait du meilleur « clean-code »
- **Gemini** : pour m'épauler dans la réalisation du rapport. Le choix s'est porté sur Gemini car je trouve que ces tournures de phrases sont « appréciables ». Il est clair que j'ai rarement copier-coller un paragraphe entier, j'essaie toujours de garder ma patte et mes sens de phrases au plus possible

Voici les pours et les contres de cette méthode :

- **Contre** :

- **Mise ne place d'un environnement** : Pour travailler durablement avec des IA, il est important de mettre en place son environnement. En effet, aujourd'hui, le standard consiste à configurer des systèmes d'agents spécialisés capables d'exécuter des tâches précises ou d'automatiser des processus comme la gestion des commits. **C'est un vrai travail en soi** : il faut apprendre à structurer cet écosystème technique pour qu'il soit un réel atout et non un frein. On remarque dans mon projet que cette étape n'a pas été faite, on peut prendre comme exemple mes commits : ils n'étaient pas du tout atomiques et cela peut poser un problème dans le temps. J'aurais très bien pu configurer ma session pour que Claude fasse des commits à chaque fois qu'il touche à 4 lignes de code, mais j'aurais **manqué de temps**.
- **Détachement du code** : En utilisant l'IA, il est facile de se détacher complètement du code. En effet, il est facile de laisser l'IA rédiger tout son code sans même y jeter un coup d'œil. Cela pose un gros problème concernant :
 1. La scalabilité de l'application : L'IA peut générer du code qui n'est en aucun cas optimisé. Le code peut fonctionner pour une centaine de personnes, mais pas pour 1000.
 2. Code incompréhensible : L'IA rédige du code souvent compliqué à comprendre pour le commun des mortels, de plus, il omet souvent de mettre des commentaires.
 3. Incompréhension architecturale : En se détachant du code, on finit souvent par ne même plus comprendre comment l'application en elle-même fonctionne.
 4. Aucun apprentissage : Si on ne s'intéresse pas au code, si on ne pose pas de questions, l'apprentissage est inexistant.
- **Design** : Dans ce projet, mes maquettes sont complètement passées à la trappe. Ici il est surtout question de temps. J'ai préféré me concentrer sur la logique métier et les fonctionnalités de PRALLO. Si on s'intéresse plus globalement au design, on remarque souvent que les designs réalisés via IA sont monotones, répétitifs, et manquent de touche « humaine » à mon goût. Cette partie me touche un peu plus car le design et le frontend sont les parties qui m'intéressent.
- **Sécurité** : En réalisant du code via IA, on peut facilement omettre la partie sécurité d'une application. Je pourrais peut-être faire le parallèle avec les frameworks utilisés dans le web d'aujourd'hui. On ne comprend pas tout, mais on fait confiance et on croise les doigts.

- **Pour**

- **Rapidité** : L'IA permet de créer des applications en un rien de temps. Alors oui, peut-être que le code n'est pas forcément optimisé, mais il permet d'avoir une « base » de code sur laquelle on peut travailler plus tard.
- **Fonctionnalité** : Claude code m'a fourni des solutions auxquels je n'avais pas pensé. Je pense notamment à la façon de gérer des fichiers. Je n'avais pas trop d'idée de comment le faire de manière « intelligente », mais Claude Code a proposé une idée vraiment sympa et fonctionnelle.
- **Apprentissage** : Si on s'intéresse au code généré, c'est une mine d'or d'apprentissage. En effet, si on pose des questions, qu'on lui demande d'ajouter des commentaires, il s'agit d'un outil d'apprentissage formidable, surtout en ce qui concerne le code.
- **Analyse** : Si on dédit la partie code, on a beaucoup plus de temps pour s'intéresser à autre chose : fonctionnalité, maquettes, architecture.

3.6 Liste des documents fournis

Les documents/liens fournis avec le rapport du TPI sont les suivants :

- **Les user story au format pdf**
- **Un lien vers le repo github**
- **Le journal de travail au format pdf**

4 Conclusions

4.1 Objectifs

A le rendu du TPI, j'ai le sentiment d'avoir remplis la grande majorité des objectifs et de m'être donné à fond pour essayer de les atteindre. Les quelques bugs sur l'environnement de staging sont minimes à la première impression. De plus les problèmes que j'ai rencontré lors de la réalisation du TPI n'ont pas impacté mon rendu final. J'ai aussi le sentiment que les échanges avec les différents partis se sont très bien passé et un sentiment de confiance a pu facilement s'installer. En terme de gestion de projet Agile, l'utilisation de GitProject et de Git m'a permis de respecter les délai et de rester au clair avec les objectif.

4.2 Difficultés particulières

J'ai eu un peu de peine à intégrer l'authentification via MSAL même en utilisant l'IA. De plus l'environnement de staging ne fonctionne pas encore.

4.3 Suites possibles pour le projet

Concernant PRALLO

- Monsieur Sonnet, un professeur à l'ETML est en train de rajouter un champ dans le token renvoyé par Microsoft, il s'agit d'un champ « rôle ». Pour PRALLO cela s'avère super intéressant car on pourrait directement venir chercher le rôle dans le token et ne plus du tout gérer cette partie. Il y aurait des rôles « doyen », « élève » directement implémentés dans le token. On devrait juste garder les permissions que l'on donne au rôle, mais c'est tout !
- PRALLO est conçu pour être adaptable, on pourrait donc très bien l'utiliser pour gérer les TPI par exemple ? Il suffirait d'ajouter un rôle « Expert » et d'un peu modifier les fonctions de répartitions et le tour serait joué !
- Le Design devrait être retouché complètement. C'est mon avis mais les applications de type « Dashboard » se ressemblent un peu toutes et PRALLO en fait partie, malheureusement, dans l'état actuel.

4.4 Avis personnel

Ce projet était super intéressant. J'ai pu apprendre beaucoup de choses que l'on n'avait pas vues en cours concernant le backend. Notamment les middlewares que nous n'avions pas forcément abordés. J'ai aussi pu apprendre à réaliser une application adaptable. La gestion de projet agile est quelque chose que je ne comprenais pas encore à 100%, maintenant, j'ai l'impression d'avoir compris le fondement même de cette pratique. Comme j'ai envie de travailler dans le frontend, je me suis vite retrouvé à travailler sur les maquettes et à faire de l'analyse et j'ai beaucoup aimé.

5 Annexes

5.1 Sources – Bibliographie

Documentation officielle

- AdonisJS v6, [AdonisJS docs - Everything you need to get building - AdonisJS Documentation](#)
- Vue 3, [Vue.js - The Progressive JavaScript Framework | Vue.js](#)
- Pinia, [Pinia | The intuitive store for Vue.js](#)
- Vue Router, [Vue Routing](#)
- Vite, [Vite | Next Generation Frontend Tooling](#)
- Cypress, [Cypress testing solutions | Cypress Documentation | Cypress Documentation](#)
- Japa (AdonisJS test runner), [Introduction \(Guides\) | Japa Documentation](#)
- MSAL Browser, [About MSAL Browser - Microsoft Authentication Library for JavaScript | Microsoft Learn](#)
- Microsoft Azure Blob Storage, [Azure Blob Storage | Microsoft Azure](#)
- VineJS (validation), [Validation \(Basics\) - AdonisJS Documentation](#)
- Axios, [API reference | axios | Promise based HTTP client](#)

- Docker, [Docker Docs](#)
- PM2, [PM2 - Home - Advanced Node.js Process Manager & Monitoring](#)

Algorithme

- Algorithme hongrois, https://en.wikipedia.org/wiki/Hungarian_algorithm

5.2 ***Figures***

Figure 1 : Environnement de développement	11
Figure 2 : environnement de staging	13
Figure 3 : Diagramme d'état	13
Figure 4 : diagramme de flux	15
Figure 5: MCD	22
Figure 6 : MLD	24
Figure 7 : SSH avec pm2.....	27
Figure 8 : tests backend	31
Figure 9 : tests avec cypress	31
Figure 10 : détail d'une erreur sur les tests backend	32
Figure 11: planification	33

5.3 Journal de travail

Journal de travail						
Auteur: Minger Théo			Projet: TPI			
Temps total: 87 heures 0 minutes			Date: 07.05.2026 au 01.06.2026			
		Temps				
Semaine	Jour	h.	min.	Type	Description	Remarques/problèmes
19	07 mai		40 min	Analyse	Début du TPI -> discussion avec l'expert concernant : le cahier des charges, directives du TPI, défense du TPI	
19	07 mai		20 min	Analyse	Relecture du cahier des charges + comprehension détaillé de celui-ci	
19	07 mai		2 h 00 min	Documentation	Réalisation de la partie 1. Analyse préliminaire du rapport -> J'ai écrit une ébauche du résumé et remplis la partie gestion de projet et objectif de formation. -> Création de la planification initiale	Le premier expert à demandé de faire une planification initiale avec des blocs de max 4h -> néanmoins lors de mon pré-TPI je n'ai pas fait ça -> Je dois demander demain au chef de projet
19	08 mai		20 min	Analyse	Discussion avec le chef de projet -> je dois garder ma planification initiale comme lors du pré-tpi. De plus nous avons eu une discussion concernant l'environnement de staging	utilisation de ssh + pm2 pour le serveur de staging, à mettre dans le rapport aussi
19	08 mai		2 h 20 min	Documentation	Continuation de la partie 1.Analyse préliminaire du rapport (objectif produit,formation: Résumé) et finalisation de la planification initiale	Discussion avec le chef de projet concernant la planification initiale -> on garde la planification originale comme déjà faite pendant le pré tpi
19	08 mai		20 min	Implémentation	Création des ressources github : repository et gitProject et initialisation des documents (dossier doc)	J'ai demandé dans mon premier email les pseudonyme github des experts.
19	08 mai		25 min	Analyse	Création des User story dans le porduct backlog -> les US manque encore de maquettes, de test d'acceptance. Ajout d'une DoD générique	https://github.com/users/TheoMingerInfo/projects/9
19	08 mai		20 min	Analyse	Création de quelques question pour le client concernant les choses pas clair dans le cdc	pas encore fini
19	08 mai		3 h 00 min	Analyse	Création du fichier Figma et début des maquettes -> les maquettes sont réalisé en fonction des user story. Cela m'aide à mieux comprendre l'application et me permet donc de préparer de meilleures questions pour le client. J'ai séparé l'application en 4 étape (comme dans le cdc) -> les maquettes suivent cette démarche (3 rôle, 4 états de l'application = 12 wireframe "contenant" les pages de l'application)	je dois prendre rendez-vous avec le client pour le lundi 11.05.2026 -> cela me permettra d'y voir un peu plus clair. Mes question seront disponible dans un fichier word.
20	11 mai		20 min	Absent	Retard	
20	11 mai		2 h 40 min	Analyse	Travail sur les maquettes figma : Réalisation des maquettes Professeur - préparation	J'ai pris rendez vous avec le chef de projet (client) pour discuter des fonctionnalité. Le rendez vous aura lieu le mardi 12.05.2026 à 9h50
20	11 mai		2 h 15 min	Analyse	Travail sur les maquettes figma : Réalisation des maquettes Doyen - préparation	Une fois que toute l'application est designé en état "Préparation", les autres états sont censés prendre moins de temps comme il suffit d'adater les maquettes à l'état -> les seules pages à faire de "0" seront celles des élèves
20	12 mai		2 h 30 min	Analyse	Travail sur les maquettes figma : Réalisation d'une bonne partie des maquettes élèves et professeur	
20	12 mai		30 min	Analyse	Discussion avec le client concernant des fonctionnalité de Prallo	La feuille des questions est disponible dans la doc. Elle ne représente pas toutes les questions posé lors de l'interview mais offre une vue d'ensemble du genre de questions que j'ai posé. Le resultat de cette discussion impacte le cdc et donc sera disponible dans la partie 2. du rapport : Analyse préliminaire
20	12 mai		2 h 30 min	Analyse	Travail sur les maquettes figma : finalisation des maquettes	
20	12 mai		30 min	Analyse	Travail sur les user stories	https://github.com/users/TheoMingerInfo/projects/9/views/1
20	13 mai		2 h 40 min	Analyse	Correction de quelques maquettes figma et continuation des users story	https://github.com/users/TheoMingerInfo/projects/9/views/1
20	13 mai		20 min	Analyse	Discussion avec Mr Melly, un professeur de l'ETML qui gère le site p2etml.	Mon application a beaucoup de ressemblance avec l'application p2etml, j'en ai donc profité pour lui poser quelques questions sur la gestion de rôle. Il m'a parlé de "rbac". Le lien suivant me sera utile lors de la réalisation : https://dev.to/slickdev_raphael/role-based-access-control-in-adonis-js-with-npm-role-acl-4jg3
20	13 mai		20 min	Analyse	Discussion avec le Product Owner / chef de projet concernant le format des users story	Ici j'ai eu quelques problème concernant le format des US en effet, comme PRALLO utilise des rôle et des état de l'application, cela me donne comme un peu une matrice des choses possible par rôle/état de l'application, je dois donc trouver un moyen de ne pas me répéter dans les US tout en expliquant toutes les possibilités. J'ai une ébauche sur mon projet git, que je montrerai au Product Owner le lundi 18.05 pour être sûr de partir sur la bonne route
20	13 mai		1 h 55 min	Analyse	Ajout de toutes les users story en lien avec les fonctionnalité	J'ai créé toutes les US mais je dois mainteant les remplir

				J'ai travaillé toute la journée sur les user story	Il est important de réaliser de bonne user story pour ensuite les convertir avec l'IA c'est pourquoi cela me prends autant de temps, de plus, il y en a énormément et certaines sont compliquées à écrire ou répétitives, on peut facilement se perdre si on est pas attentif
21	18 mai	5 h 00 min	Analyse		
21	18 mai	15 min	Analyse	Vérification avec le Chef De Projet /Product Owner concernant le format des user story	En effet comme le projet a plusieurs rôles/états, je ne savais pas vraiment comment faire, nous avons opté pour des user story détaillées et séparées par rôle
21	19 mai	4 h	Analyse	Réalisation des user stories	J'ai fini les user story et j'ai averti le chef de projet pour qu'il puisse les analyser
21	19 mai	2 h	Documentation	Réalisation de la partie 2. Analyse	J'ai commencé à écrire la partie 2 du rapport. Concernant l'analyse fonctionnelle les user story se retrouvent en annexe à cause du nombre de pages. De plus j'ai ajouté des schémas pour montrer l'architecture des environnements.
21	20 mai	4 h 30 min	Documentation	J'ai continué le rapport	J'ai ajouté la planification en brut, je compte en discuter cette après-midi avec mon chef de projet
21	20 mai	45 min	Analyse	Discussion avec le chef de projet concernant un compte client et l'implémentation des rôles/permissions + revue des user story	Le User story ont été "revus" avec le chef de projet et ont été acceptés, je peux donc caser cette coche dans ma DoD
21	21 mai	3 h 45 min	Documentation	Ajout de différents schémas et continuation du rapport 2. Analyse	J'ai continué sur ma lancée et j'arrive à la fin de la partie 2. Analyse
21	21 mai	1 h 30 min	Documentation	Fin de la partie 2. Analyse du rapport	
21	21 mai	1 h 30 min	Implémentation	J'ai créé un fichier md pour réaliser le prompt "bullet" de l'application, c'est-à-dire que je renseigne client sur les fichiers disponibles et où ils se trouvent, et je lui laisse tout faire d'un coup, on verra le résultat !	J'ai donc créé une branche develop pour respecter le git flow
				Le résultat est assez impressionnant.	Le résultat du back-end est vraiment intéressant, néanmoins, le front-end ne correspond pas à 0% à mes maquettes. Il est très important maintenant de respecter le gitflow et les pratiques en terme de gestion de code pour ne pas me perdre. L'IA à néanmoins fait beaucoup "d'erreurs". On a par exemple vu avec Mr Carrel que la base de données n'était pas vraiment clean et ne correspondait pas à un travail professionnel, mais la base était là !. Il faut donc maintenant prendre toutes les user story et regarder si les conditions sont remplies. Je le fais au moyen de "test manuel" même si automatiser tout cela avec des tests serait plus intelligent, je le ferais plus tard par exemple pour les permissions. J'ai d'ailleurs aussi regardé les tests qu'il m'avait fournis comme demandé, et ils sont plutôt exhaustifs c'est cool ! Juste les tests Cypress qui ne fonctionnent pas comme l'authentification via Azure ne fonctionne pas !
21	22 mai	3 h	Implémentation		
				J'ai travaillé sur le code fourni, travail sur les features et bugfixes de l'application PRALLO	J'ai commencé à reprendre le code généré par l'IA petit à petit et je commence à le fix. J'ai mes user story d'un côté, l'application de l'autre, et le visual studio code encore à côté. J'ai créé ma première branche "Feature" pour fixer le fait que l'on ne pouvait pas ajouter de fichier à la création d'un projet. Pour la suite du projet je vais fonctionner comme cela. ! Rappel pour moi : je dois discuter d'une pull request avec le chef de projet avant la fin du TPI
21	22 mai	3 h	Implémentation		
22	26 mai	3 h	Implémentation	Travail sur les bugfixes et les features manquantes dans l'application PRALLO	J'ai enfin mis en place l'utilisation du git flow c'est-à-dire -> j'ai créé les branches Features et Bugfix quand j'avais des modifications à faire. J'ai réalisé plusieurs bugfixes et features que l'on peut retrouver dans mon github
22	26 mai	1 h 30 min	Implémentation	Travail sur la fonctionnalité de répartition de PRALLO	PRALLO répartie des projets - élèves - professeurs. Au début du projet je m'étais déjà renseigné sur des algorithmes de tri de paires avec satisfaction, j'étais tombé sur l'algorithme hongrois. Je me suis donc renseigné sur comment celui-ci fonctionnait et j'ai donc voulu l'implémenter dans le projet. Seul problème, il fonctionne uniquement sur des matrices 2D, alors que nous recherchons à répartir élèves - projets - professeurs. La solution utilise l'algorithme hongrois pour répartir les élèves - Projets et attribue de manière un peu déterministe les professeurs capables de réaliser les projets
				Revue du code avec le chef de projet et review d'une pull request	Le chef de projet a réalisé le parcours utilisateurs de l'application PRALLO. Beaucoup de fonctionnalités sont présentes et il me manque peu de retouche dans le code. Une review de pull request a été réalisée et nous avons discuté de comment réaliser le gitflow en utilisant l'IA. En effet il est compliqué de réaliser des commits atomiques quand on utilise l'IA comme je le fais. Il est possible d'avoir un environnement préparé avec l'IA, c'est à dire qu'il fera des commits etc... Néanmoins ça n'a pas été mon choix. Résultat, des commits un peu longs et qui touchent à plusieurs choses (non atomiques)
22	26 mai	55 min	Analyse		
22	26 mai	35 min	Documentation	Revue du journal de travail et du rapport	
22	27 mai	3 h	Implémentation	travail sur les fonctionnalités de PRALLO	J'ai ajouté les dernières fonctionnalités de PRALLO et corrigé les bugs

[illegible]

5.4 Manuel d'utilisation

PRALLO — Project Allocator ETML

Application web de gestion et répartition de projets de fin d'année pour l'ETML.

Prérequis de développement

Outils nécessaires

Outil	Version recommandée	Utilité
Node.js	22 LTS	Backend et frontend
Docker Desktop	Dernière version	Base de données MySQL en local
Git	Dernière version	Gestion du code source

Installation et lancement en local

1. Base de données

Démarrer le conteneur MySQL et phpMyAdmin :

```
cd src
docker-compose up -d
```

La base de données est accessible sur `localhost:3306`.
phpMyAdmin est accessible sur `http://localhost:8080`.

2. Backend

```
cd src/backend/PRALLO_api
npm install
node ace migration:run
node ace db:seed --files database/seeder/permissions_seeder.ts
node ace serve --hmr
```

L'API tourne sur `http://localhost:3333`.

3. Frontend

```
cd src/frontend/PRALLO_front
npm install
npm run dev
```

Le frontend tourne sur `http://localhost:5173`.

Variables d'environnement

À compléter par les valeurs de votre environnement.

Backend — `src/backend/PRALLO_api/.env`

```
TZ=UTC
PORT=3333
HOST=localhost
LOG_LEVEL=info
APP_KEY=                                # Générer avec : node ace generate:key
NODE_ENV=development

# Base de données
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=prallo
DB_PASSWORD=prallo
DB_DATABASE=prallo

# Azure AD
AZURE_CLIENT_ID=                        # Client ID de l'application Azure
AZURE_TENANT_ID=                        # Tenant ID Azure

# Mode dev : permet de se connecter sans Azure (true uniquement en développement)
DEV_BYPASS_AUTH=false

# Azure Blob Storage (pour les fichiers de projets)
AZURE_STORAGE_CONNECTION_STRING=
AZURE_STORAGE_CONTAINER=prallo-files
```

Frontend — `src/frontend/PRALLO_front/.env`

```
VITE_API_URL=http://localhost:3333      # URL du backend
VITE_AZURE_CLIENT_ID=                   # Même valeur que le backend
VITE_AZURE_TENANT_ID=                   # Même valeur que le backend
VITE_REDIRECT_URI=http://localhost:5173 # URL de retour après login Azure
VITE_DEV_BYPASS=false                   # true pour activer le bypass dev
```

Utilisation de l'application

Connexion

L'authentification se fait via les comptes **Eduvaud** (Microsoft Azure AD). Cliquer sur "Se connecter avec Eduvaud" redirige vers la page de connexion Microsoft.

En mode développement (`VITE_DEV_BYPASS=true`), un menu de bypass permet de se connecter directement avec un compte de test sans passer par Azure.

Rôles

Rôle	Accès
Doyen	Accès complet — gère l'état, les participants, la répartition
Professeur	Crée et gère ses projets
Étudiant	Consulte les projets et vote selon l'état actuel

Invité	Accès limité selon l'état
---------------	---------------------------

États de l'application

L'application passe par 5 états gérés par le doyen :

1. **Préparation** — Les professeurs créent leurs projets
 2. **Repérage** — Les projets sont visibles par tous
 3. **Vote** — Les étudiants votent pour leurs projets préférés (max 3)
 4. **Répartition** — Le doyen calcule et ajuste la répartition
 5. **Publication** — La répartition est publiée et visible par tous
-

Staging

Initialisation de la base de données

Lancer uniquement les migrations :

```
node ace migration:run --force
```

Lancer uniquement le seeder des permissions (rôles, états et droits) :

```
node ace db:seed --files database/seeder/permissions_seeder.js
```

Note : sur le serveur, utiliser l'extension `.js` (fichiers compilés) et non `.ts`.

Premier utilisateur — rôle Doyen

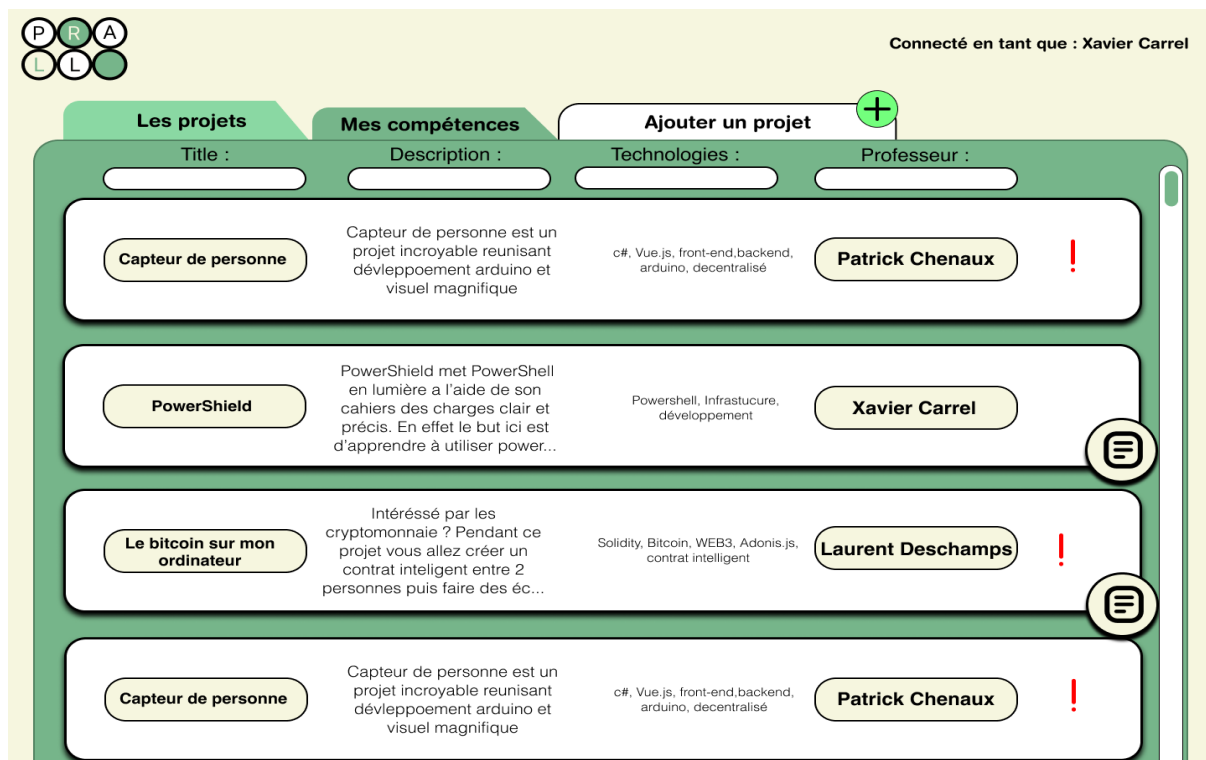
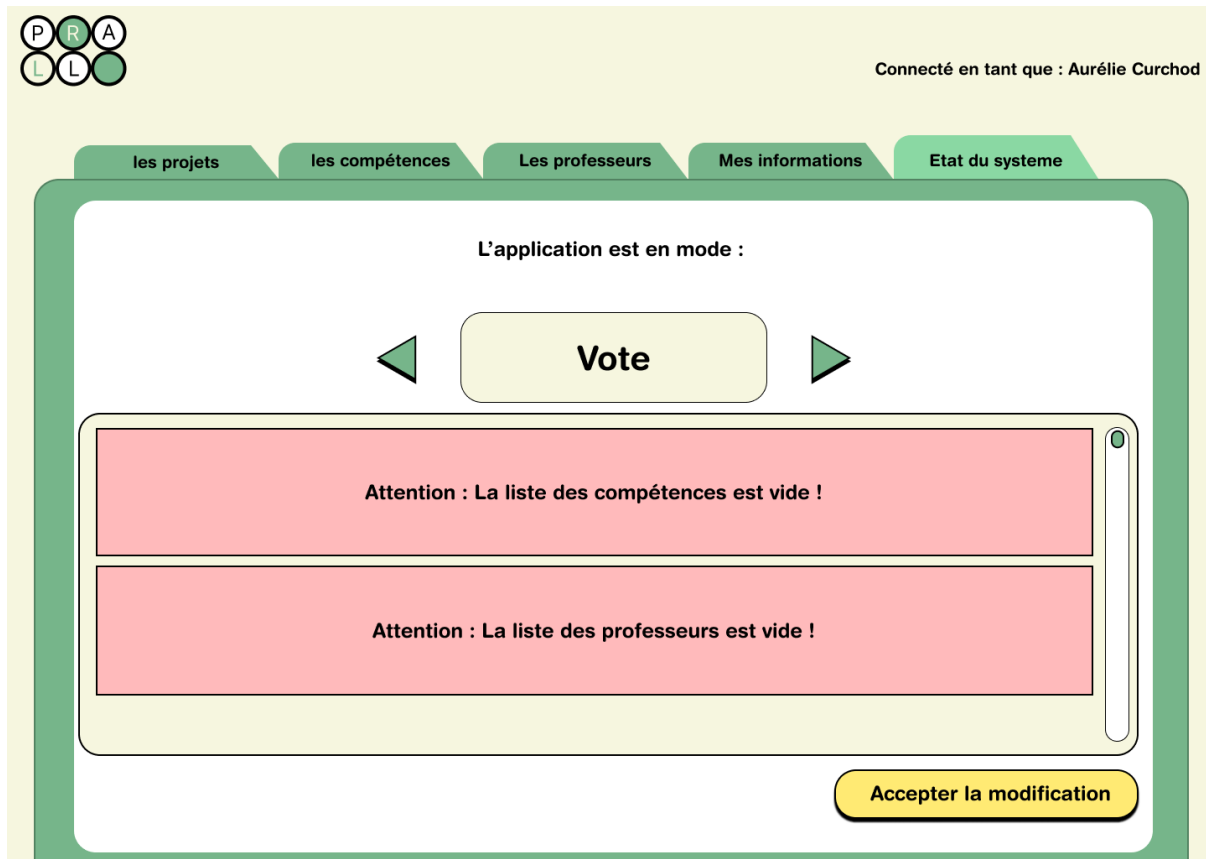
La première personne qui se connecte via Azure AD est automatiquement assignée au rôle **doyen**. Tous les utilisateurs suivants reçoivent le rôle **invité** par défaut. Le doyen peut ensuite modifier les rôles depuis la page Participants.

Redémarrer le backend

```
pm2 restart PRALLO_API  
pm2 logs PRALLO_API --lines 50
```

5.5 Maquettes

Toutes les maquettes ne sont pas disponibles ici. Si cela vous intéresse, vous les trouvez toutes dans le dossier doc du github



PLA

Connecté en tant que : Xavier Carrel

Rechercher :

Technologies :

C# +

C# - Front-end - Cryptomonnaie -

Ajouter

Le bitcoin sur mon ordinateur

Intéressé par les cryptomonnaie ? Pendant ce projet vous allez créer un contrat intelligent entre 2 personnes puis faire des éc...

Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent

!

PLA

Connecté en tant que : Xavier Carrel

Titre :

Description :

Technologies :

Document facultatifs :

Professeur :

Créer le projet

d'apprendre à utiliser power...

Le bitcoin sur mon ordinateur

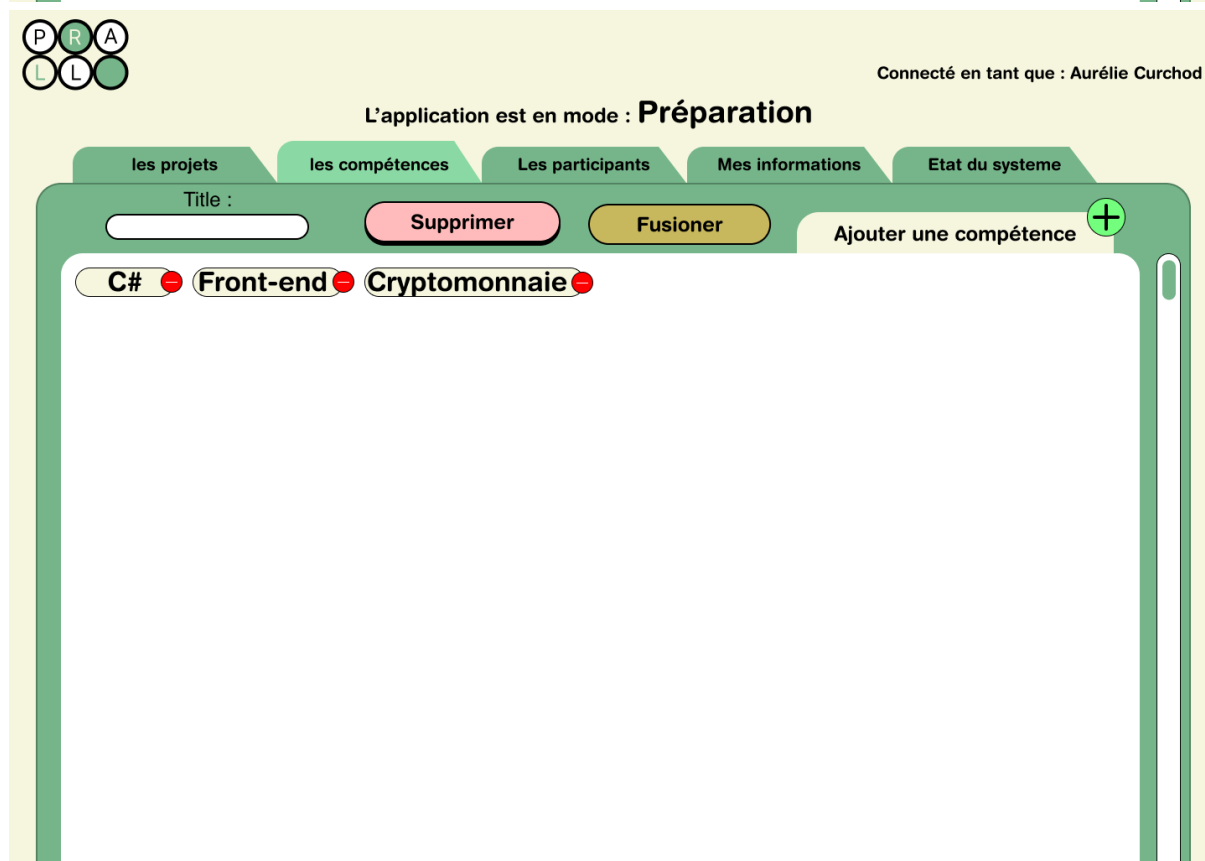
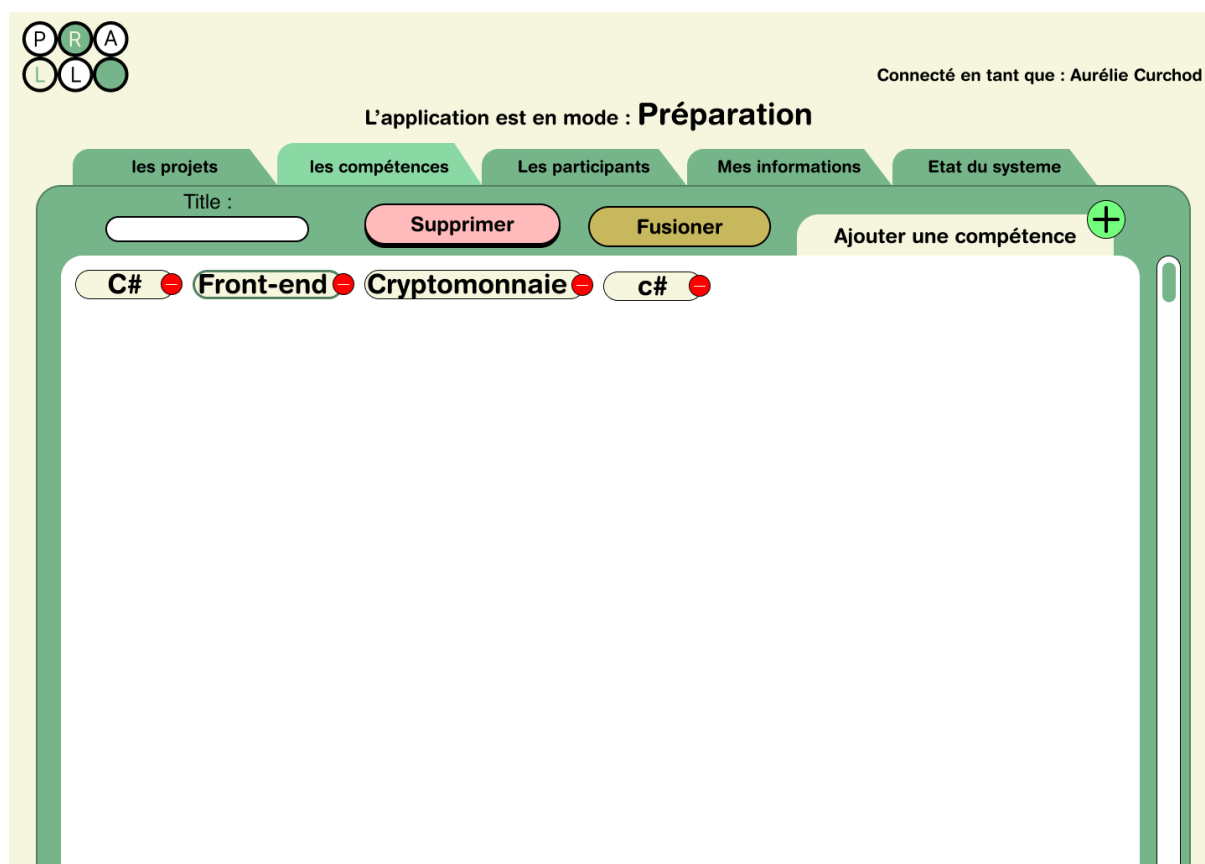
Intéressé par les cryptomonnaie ? Pendant ce projet vous allez créer un contrat intelligent entre 2 personnes puis faire des éc...

Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent

!

The screenshot shows a web application interface for managing projects. At the top left is a logo with the letters P, R, A, L, L. At the top right, it says 'Mes information →' and 'Connecté en tant que : Xavier Carrel'. Below this is a header with three input fields: 'Title :', 'Description :', and 'Technologies :'. The main content area is a form for a project titled 'Capteur de personne'. It has three sections: 'Titre :', 'Description :', and 'Technologies :'. The 'Titre' section contains the text 'Ce projet doit être réalisé !'. The 'Description' section contains the text 'Capteur de personne est un projet incroyable reunisant dévelloement arduino et visuel magnifique'. The 'Technologies' section contains the text 'c#, Vue.js, front-end, backend, arduino, decentralisé'. Below these sections is a 'Document facultatifs :' section with a large empty text area. At the bottom of the form, there is a small section with the title 'Le bitcoin sur mon ordinateur' and a description: 'cryptomonnaie / Pendant ce projet vous allez créer un contrat intelligent entre 2 personnes puis faire des éc...'. To the right of this description is a list of technologies: 'Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent'. A red exclamation mark icon is visible on the right side of the form.

The screenshot shows a web application interface for managing projects. At the top left is a logo with the letters P, R, A, L, L. At the top right, it says 'Connecté en tant que : Aurélie Curchod'. Below this is a header with the text 'L'application est en mode : Préparation'. There are five tabs: 'les projets', 'les compétences', 'Les participants', 'Mes informations', and 'Etat du systeme'. The 'les compétences' tab is selected. Below the tabs is a form with a 'Title :' input field and three buttons: 'Supprimer', 'Fusionner', and 'Ajouter une compétence'. The 'Supprimer' button is highlighted. Below the buttons is a large empty text area. At the bottom of the form, there is a 'Supprimer' button. The main content area is a confirmation dialog with the text 'Etes vous sûr de vouloir supprimer les compétences suivantes'. Below this text is a list of competencies: 'Front-end'. A red minus sign icon is next to 'Front-end'. A 'Supprimer' button is at the bottom right of the dialog.



 Connecté en tant que : Aurélie Curchod

L'application est en mode : **Préparation**

les projets | les compétences | **Les participants** | Mes informations | Etat du système

Nom :	Technologies :	Nmbr de projets :	Rôle :
Patrick Chenaux	c#, Vue.js, front-end, backend, arduino, décentralisé, cryptomonnaie, bitcoin	3 ▼	Professeur ▼
Matias Denis			Etudiant ▼
Mr Sally	Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent	0 ▼	Doyen ▼
Jonhatan Melly			Invité ▼

 Connecté en tant que : Aurélie Curchod

L'application est en mode : **Préparation**

les projets | les compétences | **Les professeurs** | Mes informations | Etat du système

Titre :

Capteur de personne

Obligation de réalisation

Oui ☒ Non ☐

Document facultatifs :

Capteur de personne est un projet incroyable reunissant développement arduino et visuel magnifique

Description :

Technologies :

c#, Vue.js, front-end, backend, arduino, décentralisé

Accepter la modification

Supprimer le projet

Capteur de personne

c#, Vue.js, front-end, backend, arduino, décentralisé

P

R

A

L

L

A

Connecté en tant que : Aurélie Curchod

L'application est en mode : Préparation

les projets

les compétences

Les participants

Mes informations

Etat du systeme

Title :

Description :

Technologies :

Professeur :

Capteur de personne

Capteur de personne est un projet incroyable reunissant dévelpoement arduino et visuel magnifique

c#, Vue.js, front-end.backend, arduino, decentralisé

Patrick Chenaux

PowerShield

PowerShield met PowerShell en lumière a l'aide de son cahiers des charges clair et précis. En effet le but ici est d'apprendre à utiliser power...

Powershell, Infrastucure, développement

Xavier Carrel

Le bitcoin sur mon ordinateur

Intéressé par les cryptomonnaie ? Pendant ce projet vous allez créer un contrat intelligent entre 2 personnes puis faire des éc...

Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent

Laurent Deschamps

Capteur de personne

Capteur de personne est un projet incroyable reunissant dévelpoement arduino et visuel magnifique

c#, Vue.js, front-end.backend, arduino, decentralisé

Patrick Chenaux

P

R

A

L

L

A

Connecté en tant que : Aurélie Curchod

L'application est en mode : Préparation

les projets

les compétences

Les participants

Mes informations

Etat du systeme

Nom :

Technologies :

Nmbr de projets :

Rôle :

Patrick Chenaux

c#, Vue.js, front-end.backend, arduino, decentralisé, cryptomonnaie, bitcoin

3 ▼

Professeur ▼

Matias Denis

Etudiant ▼

Mr Sally

Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent

0 ▼

Doyen ▼

Jonhatan Melly

Invité ▼



L'application est en mode : **Préparation** Connecté en tant que : Aurélie Curchod

les projets les compétences Les participants **Mes informations** Etat du systeme

Nom : Aurelie Curchod
Nombre de projet : 0
Statut : Doyen



Connecté en tant que : Aurélie Curchod

les projets les compétences Les professeurs **Mes informations** Etat du systeme

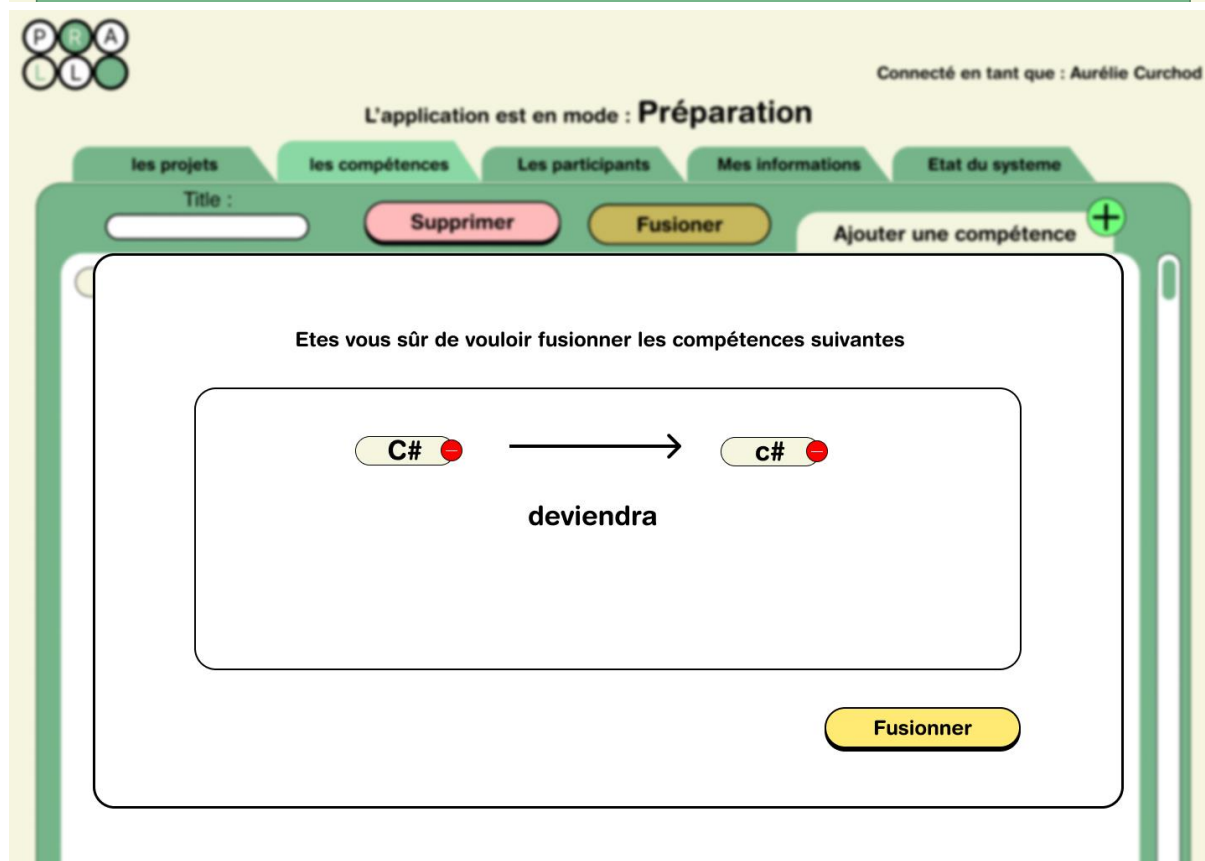
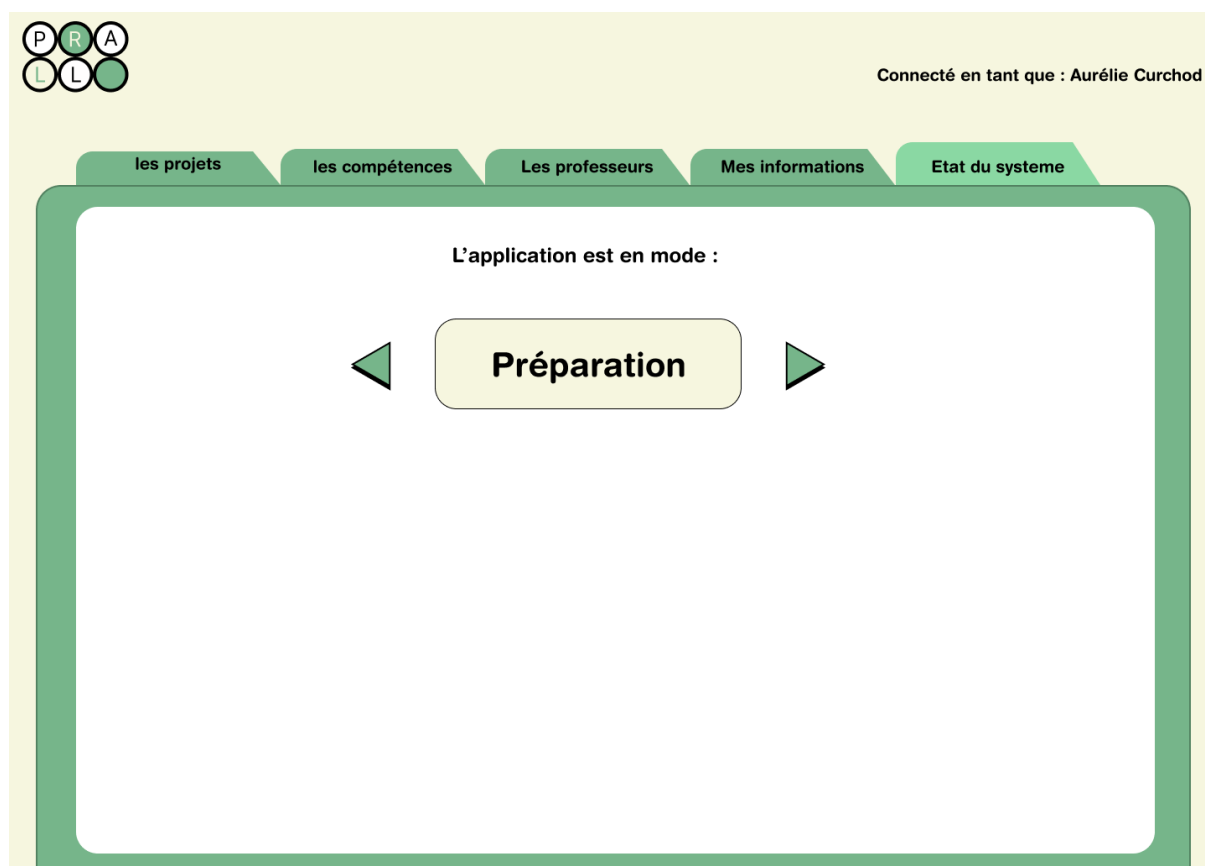
L'application est en mode :

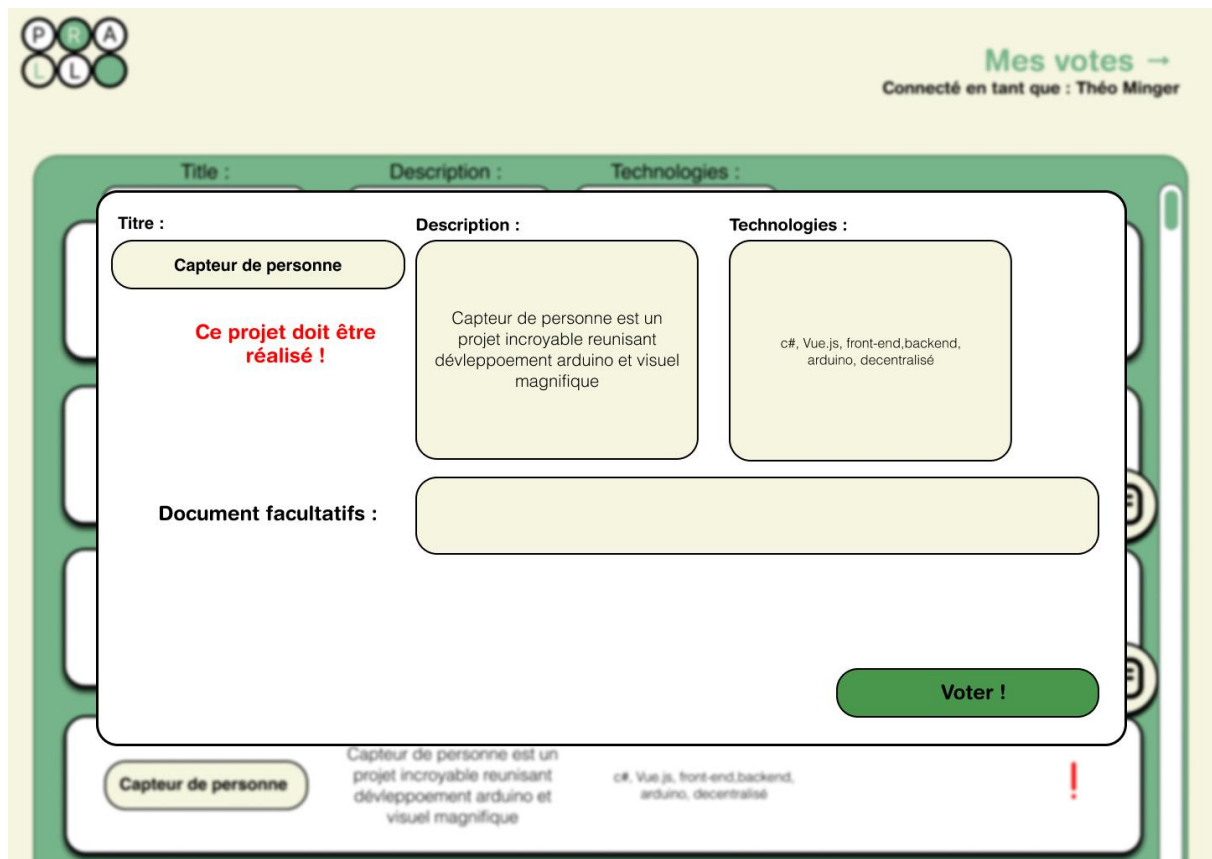
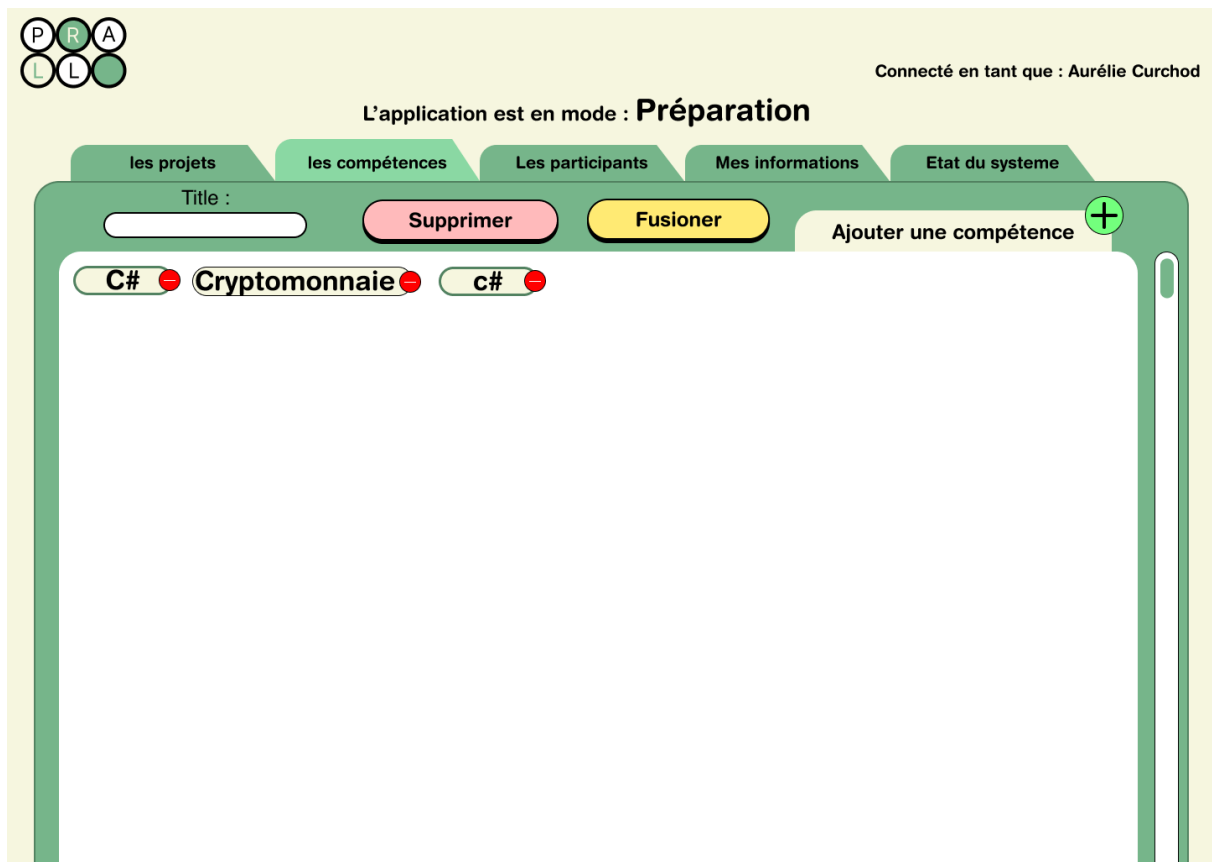
◀ **Vote** ▶

Attention : La liste des compétences est vide !

Attention : La liste des professeurs est vide !

Accepter la modification





P

R

A

L

L

Mes votes →
Connecté en tant que : Théo Minger

Title :	Description :	Technologies :	Professeur :
Capteur de personne	Capteur de personne est un projet incroyable reunisant dévelloement arduino et visuel magnifique	c#, Vue.js, front-end,backend, arduino, decentralisé	Patrick Chenaux !
PowerShield	PowerShield met PowerShell en lumière a l'aide de son cahiers des charges clair et précis. En effet le but ici est d'apprendre à utiliser power...	Powershell, Infrastucure, développement	Xavier Carrel
Le bitcoin sur mon ordinateur	Intéressé par les cryptomonnaie ? Pendant ce projet vous allez créer un contrat intelligent entre 2 personnes puis faire des éc...	Solidity, Bitcoin, WEB3, Adonis.js, contrat intelligent	Laurent Deschamps !
Capteur de personne	Capteur de personne est un projet incroyable reunisant dévelloement arduino et visuel magnifique	c#, Vue.js, front-end,backend, arduino, decentralisé	Patrick Chenaux !

P

R

A

L

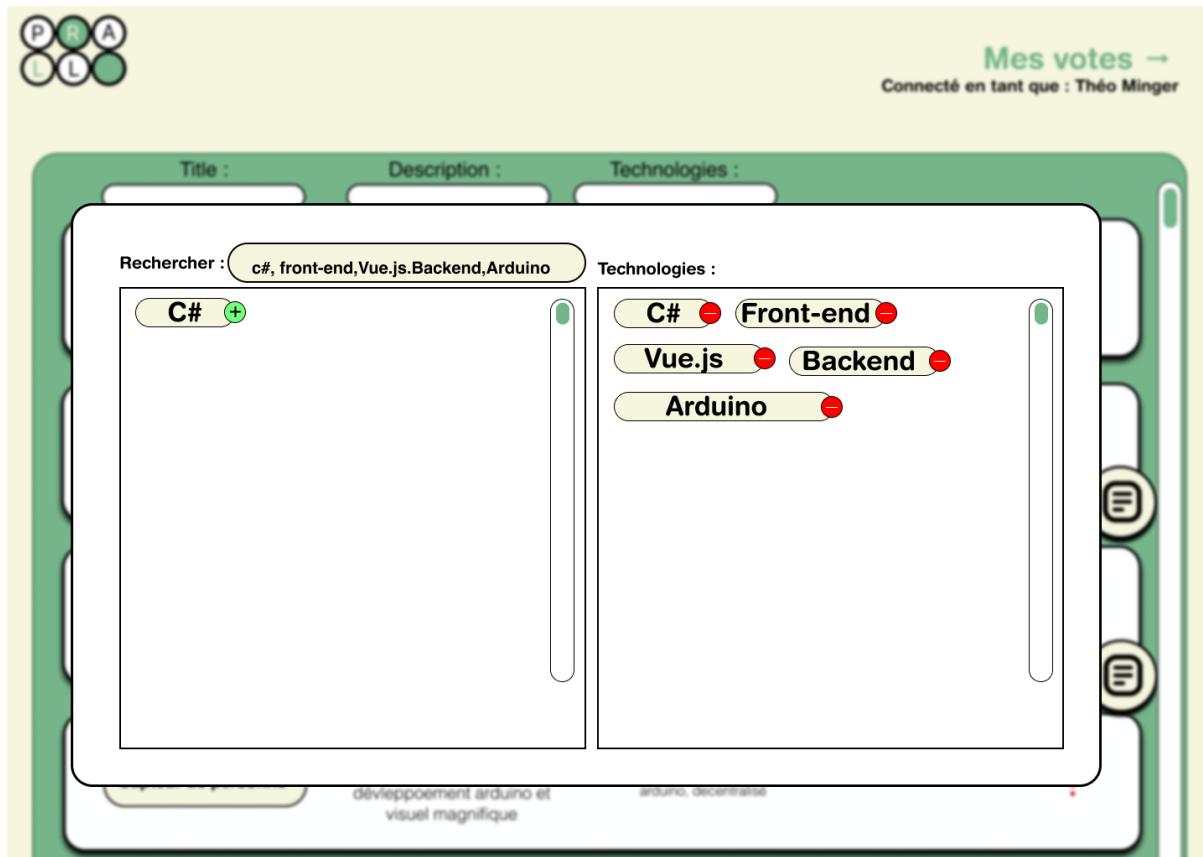
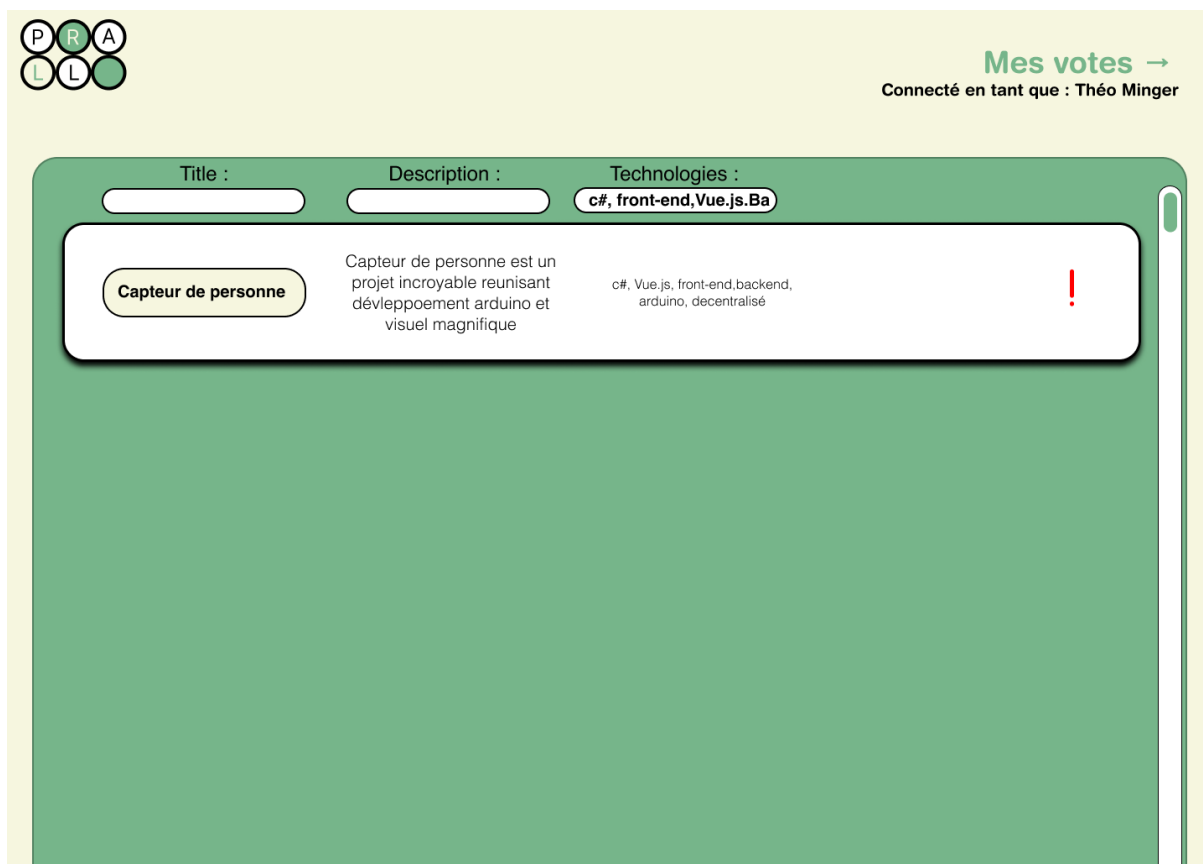
L

Les projets →
Connecté en tant que : Théo Minger

1er choix

2ème choix

3ème choix



5.6 Prompt-bullet

Je souhaite créer une application qui s'appelle PRALLO.

CONTEXTE GÉNÉRAL

Chaque année au mois de février, les étudiants de quatrième année de l'ETML reviennent de leur stage en entreprise. Il ne leur reste plus qu'un semestre à faire durant lequel se situe le TPI. En guise de préparation au TPI, les étudiants effectuent un projet d'approfondissement (P_APPRO) qui a pour but de les re-familiariser avec des méthodes de travail et/ou des technologies qu'ils n'ont pas utilisés durant une année entière, mais qu'ils vont devoir appliquer dans leur TPI. Chaque année, une des principales difficultés organisationnelles de la filière informatique est de déterminer quel étudiant fera quel projet P_APPRO avec quel enseignant. Le système « Project Allocator » (Prallo) à réaliser dans le cadre de ce TPI a pour but d'apporter une aide substantielle à la filière informatique dans ce moment compliqué.

LIVRABLES ATTENDUS

- un dossier contenant le back-end Adonis.js v6 (API REST)
- un dossier contenant le front-end Vue.js (Single-Page-Application)
- un docker compose pour la base de donnée en MySQL Le tout doit se trouver dans le dossier /src

Analyse

Je veux que tu analyse plusieurs fichier et que le résultat corresponde.

- Un rapport décrivant la démarche du projet, les architectures, les diagrammes est disponible dans le dossier /doc/Rapport_TPI_Théo_Minger.pdf
- Des User story sont disponibles dans le fichier /doc/TPI_Prallo_project_US.md
- Des Maquettes sont disponibles dans le dossier Maquettes -> /doc/Maquettes/
- Un cahier des charges est disponible dans /doc/CdC Théo Minger_signé(theo).pdf

Je veux que tu analyses tous ces fichier et dossiers pour comprendre l'application en entière. Tu dois analyser toutes les images aussi

Code

Suis mes directives concernant :

Stack

- Node v22
- Front-end : Vue.js,Pinia,Axios,GSAP si besoin,Inertia si besoin,Vite,Vitest,Cypress
- Backend : Adonis.js V6,Jappa, Docker, MySQL si des librairie sont déjà utilisé dans le framework pas besoin de télécharger des autres versions

Front-end

- Quand quelques chose se répète au niveau du front-end, crée des composant.
- Utilise Axios et Pinia quand c'est nécessaire
- intègre la connexion via compte edurvaud avec MSAL. Part du principe que j'ai toute les données concernant la connexion au services Azure (Client_id,Tenant_id, etc...), celle si doit fonctionner en développement et en Staging. De plus ajoute un moyen de contourner cette connexion quand on est en mode développement

Back-end

- Il est important que chaque rôle à des permissions en fonction de l'état de PRALLO. je veux que tu stock dans la base de donnée au format JSON ces informations. Un exemple serait 1. Rôle = Professeur 2. Etat = Préparation 3. (editProject: true, deleteProject: false). Le contrôle/attribution des permissions ce fait avant la redirection à la page

d'accueil via un middleware

- tout ce qui touche à la logique métier doit être écrit dans les modèle et non dans les contrôleur, comme son nom l'indique, le contrôleur est là pour vérifier si les conditions du modèle sont respectées.
- Crée un seeder complet qui me permet de tester l'application quand je lance `migration:refresh --seed`

est

- Dans les User story, une partie # Gestion d'erreur est disponible : je veux que pour chaque erreur possible décrite, tu crée un test unitaire à l'aide des bibliothèques fournies dans le framework. De plus crée des tests E2E pour tester un point spécifique par vue à l'aide de Cypress

sécurité

- Respecte les normes de sécurité dans tout le code